

Sayısal Görüntü İşleme

Prof Dr. Gökhan BİLGİN
Yıldız Teknik Üniversitesi
Bilgisayar Mühendisliği Bölümü

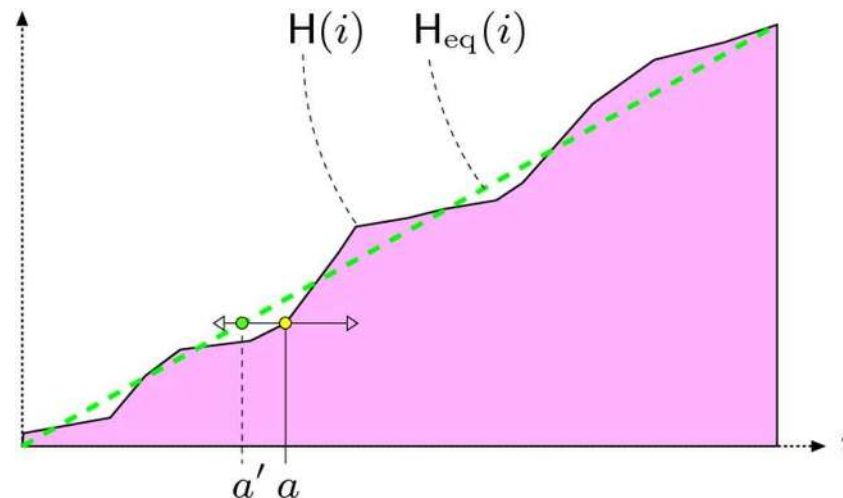
Linear Histogram Equalization

- Histogram cannot be made exactly flat – peaks cannot be increased or decreased by point operations.
- Following point operation makes histogram as flat as possible:
(assuming $M \times N$ image and pixels in range $[0, K - 1]$)

Point operation that returns
Linear equalized value of a

$$f_{\text{eq}}(a) = \left\lfloor H(a) \cdot \frac{K-1}{MN} \right\rfloor$$

Cumulative Histogram: Σ how many
times intensity a occurs



Effects of Linear Histogram Equalization

Original Image /



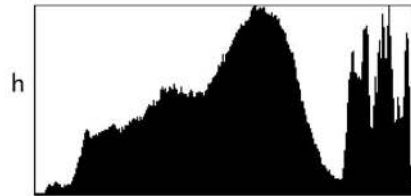
(a)



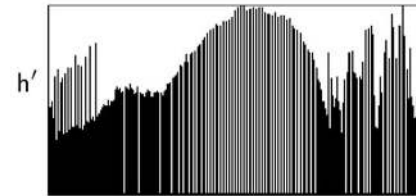
Image I' after Linear Equalization

(b)

Original histogram



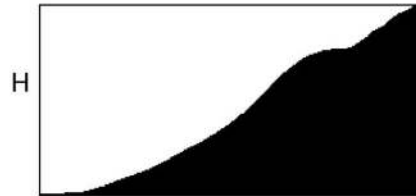
(c)



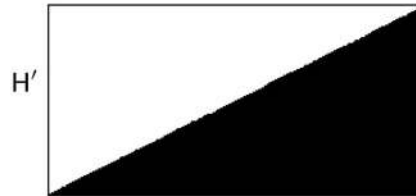
(d)

Histogram after Linear Equalization

Cumulative Histogram



(e)



(f)

Cumulative Histogram After Linear Equalization

Sample Linear Equalization Code

```
1 public void run(ImageProcessor ip) {
2     int w = ip.getWidth();
3     int h = ip.getHeight();
4     int M = w * h; // total number of image pixels
5     int K = 256;   // number of intensity values
6
7     // compute the cumulative histogram:
8     int[] H = ip.getHistogram();
9     for (int j = 1; j < H.length; j++) {
10         H[j] = H[j-1] + H[j];
11     }
12
13     // equalize the image:
14     for (int v = 0; v < h; v++) {
15         for (int u = 0; u < w; u++) {
16             int a = ip.get(u, v);
17             int b = H[a] * (K-1) / M;
18             ip.set(u, v, b);
19         }
20     }
21 }
```

← Obtain histogram of image *ip*

← Compute cumulative histogram
in place

← Get intensity value at (u,v)

← Equalize pixel intensity

Cumulative Histogram: Σ how
many times intensity *a* occurs

$$f_{eq}(a) = \left\lfloor H(a) \cdot \frac{K-1}{MN} \right\rfloor$$

Histogram Specification

- Real images never show uniform distribution (unnatural)
- Most real images, distribution of pixel intensities is gaussian
- Histogram specification
 - modifies an image's histogram into an arbitrary intensity distribution (may not be uniform)
- Image 1's histogram can also be used as target for image 2
 - Why? Makes images taken by 2 different cameras to appear as if taken by same camera

Images and Probability

Histograms can be interpreted as probabilities.

Question: If I pick a pixel from an image at random, what is the probability that the pixel has intensity i ?

Answer: $P(I(u, v) = i) = \frac{\text{\# pixels with value } i}{\text{total \# pixels}}$

Or, in terms of the histogram h :

$$P(I(u, v) = i) = h(i)/wh$$

Histogram Specification

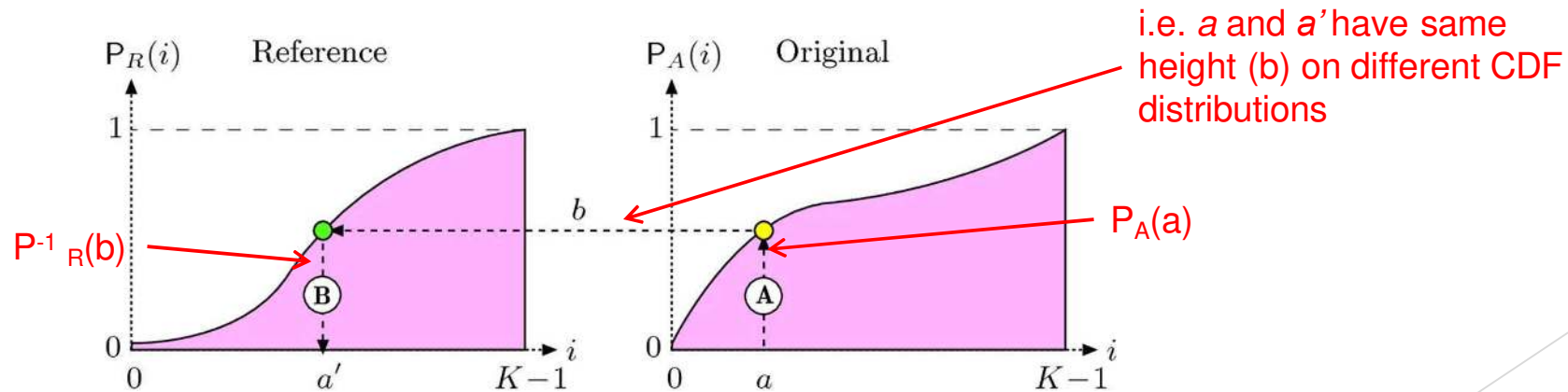
- Find a mapping such that distribution of a matches some reference distribution.i.e

$$a' = f_{\text{hs}}(a)$$

Mapping function: maps distribution on right to equivalent point (same height) On distribution on left

to convert original image I_A into $I_{A'}$ such that

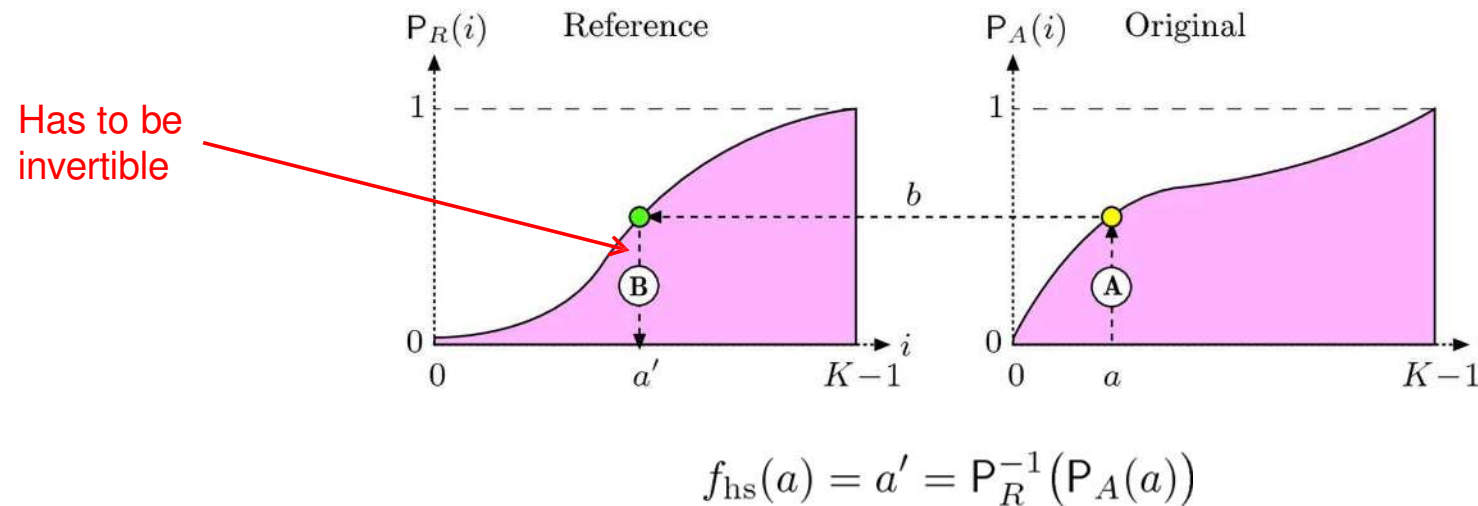
$$P_{A'}(i) \approx P_R(i) \quad \text{for } 0 \leq i < K$$



$$f_{\text{hs}}(a) = a' = P_R^{-1}(P_A(a))$$

Histogram Matching

- Prior method needed reference distribution to be invertible

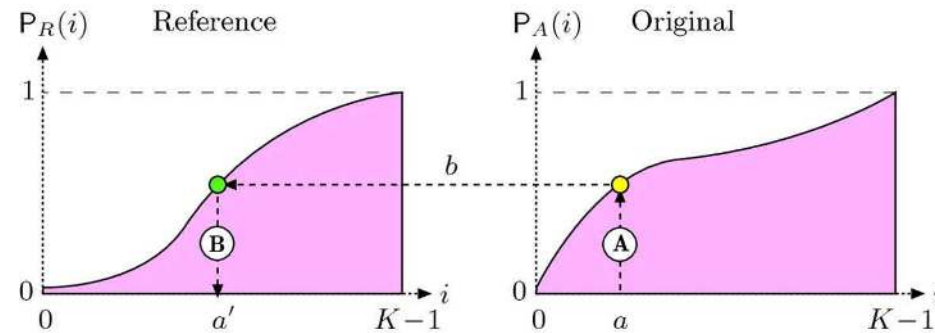


- What if reference histogram is not invertible?
- For example not invertible if histogram has some intensities that occur with probability 0? i.e. $p(k) = 0$
- Use different method called **histogram matching**

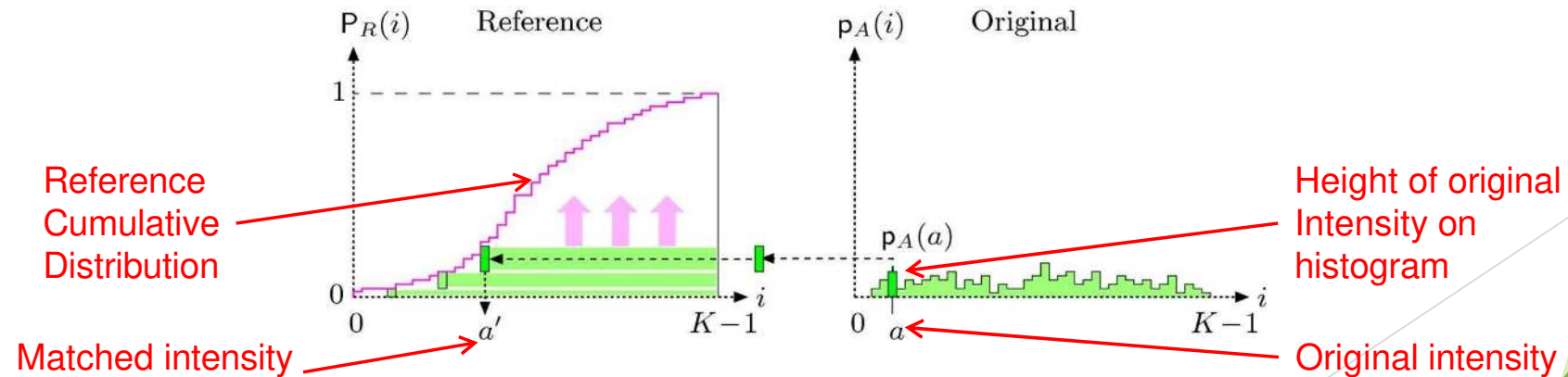
Histogram Matching

- Given two images I_A and I_B , we want to make their intensity profiles look as similar as possible.
- How?
 - “Match” their cumulative histograms H_A and H_B
- Works well for images with similar content.
- Looks bad for images with different content.

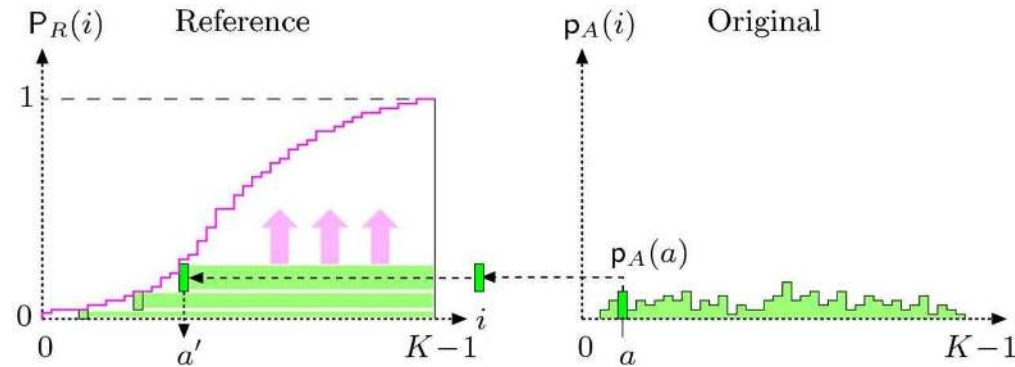
Adjusting to a Given Histogram



$$f_{\text{hs}}(a) = a' = \min \{ j \mid (0 \leq j < K) \wedge (P_A(a) \leq P_R(j)) \}$$



Adjusting to a Given Histogram

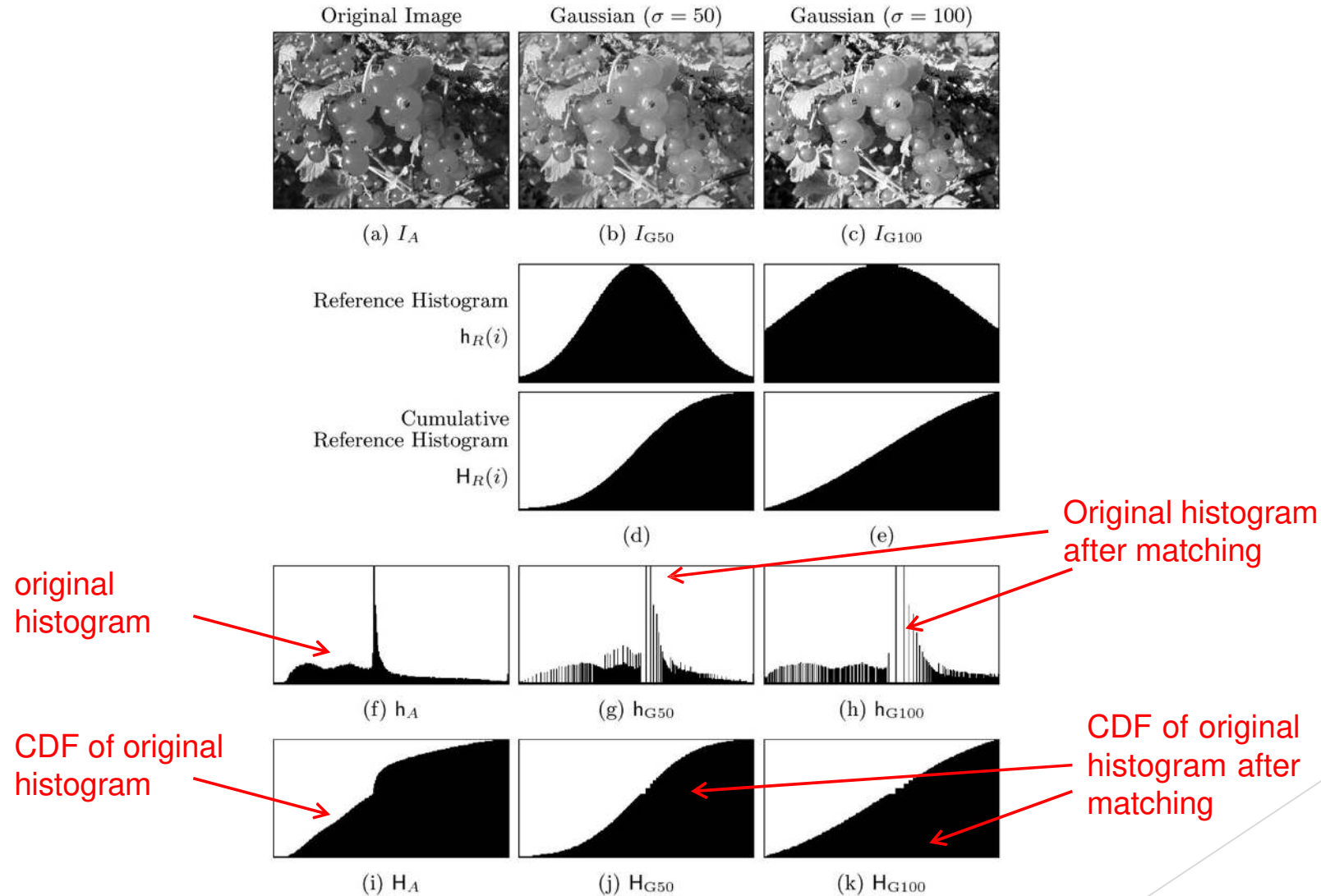


```

1: MATCHHISTOGRAMS( $h_A, h_R$ )
    $h_A$ : histogram of the target image
    $h_R$ : reference histogram (of same size as  $h_A$ )

2: Let  $K \leftarrow \text{Size}(h_A)$ 
3: Let  $P_A \leftarrow \text{CDF}(h_A)$  ▷ cdf for  $h_A$  (Alg. 5.1)
4: Let  $P_R \leftarrow \text{CDF}(h_R)$  ▷ cdf for  $h_R$  (Alg. 5.1)
5: Create a table  $f_{\text{hs}}[]$  of size  $K$  ▷ pixel mapping function  $f_{\text{hs}}$ 
6: for  $a \leftarrow 0 \dots (K-1)$  do
7:    $j \leftarrow K-1$ 
8:   repeat
9:      $f_{\text{hs}}[a] \leftarrow j$ 
10:     $j \leftarrow j-1$ 
11:   while  $(j \geq 0) \wedge (P_A(a) \leq P_R(j))$ 
12: return  $f_{\text{hs}}$ .
    
```

Adjusting to a Given Histogram



Adjusting to a Given Histogram

Target Image



(a) I_A

Reference Image

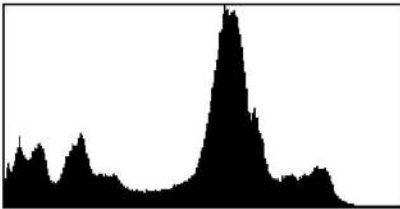


(b) I_R

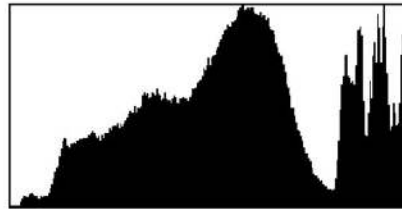
Modified Image



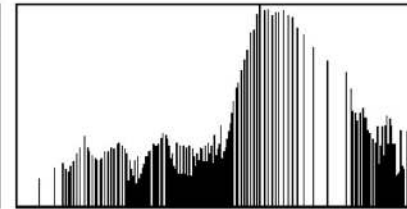
(c) $I_{A'}$



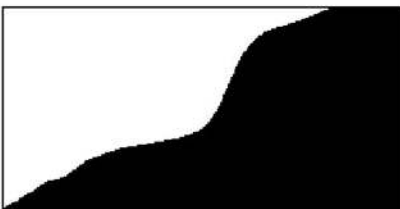
(d) h_A



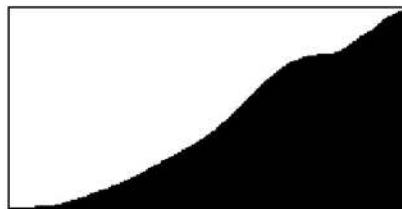
(e) h_R



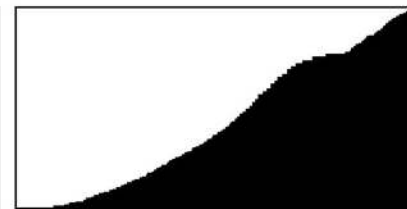
(f) $h_{A'}$



(g) H_A



(h) H_R



(i) $H_{A'}$

What Is Image Enhancement?

Image enhancement makes images more useful by:

- Highlighting interesting detail in images
- Removing noise from images
- Making images more visually appealing

Image Enhancement Examples

Images taken from Gonzalez & Woods, Digital Image Processing (2002)



Image Enhancement Examples (cont...)

Images taken from Gonzalez & Woods, Digital Image Processing (2002)

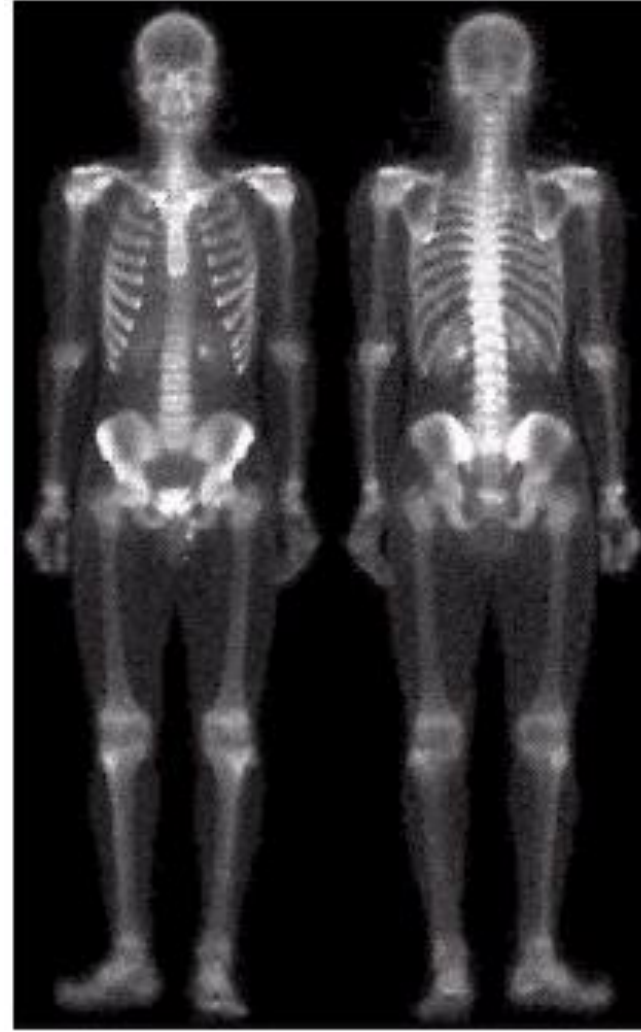
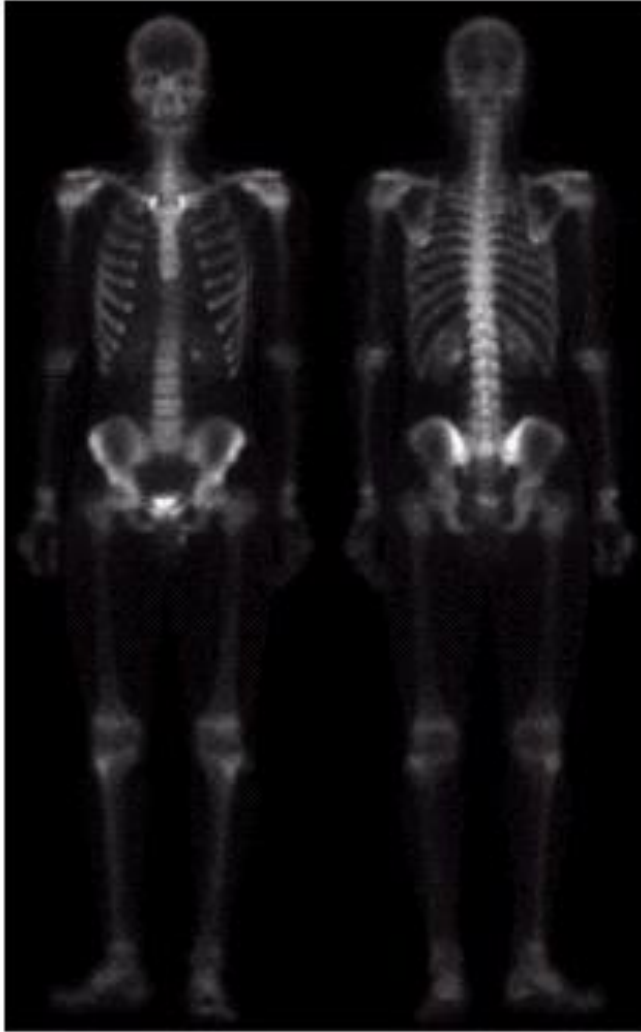


Image Enhancement Examples (cont...)

Images taken from Gonzalez & Woods, Digital Image Processing (2002)

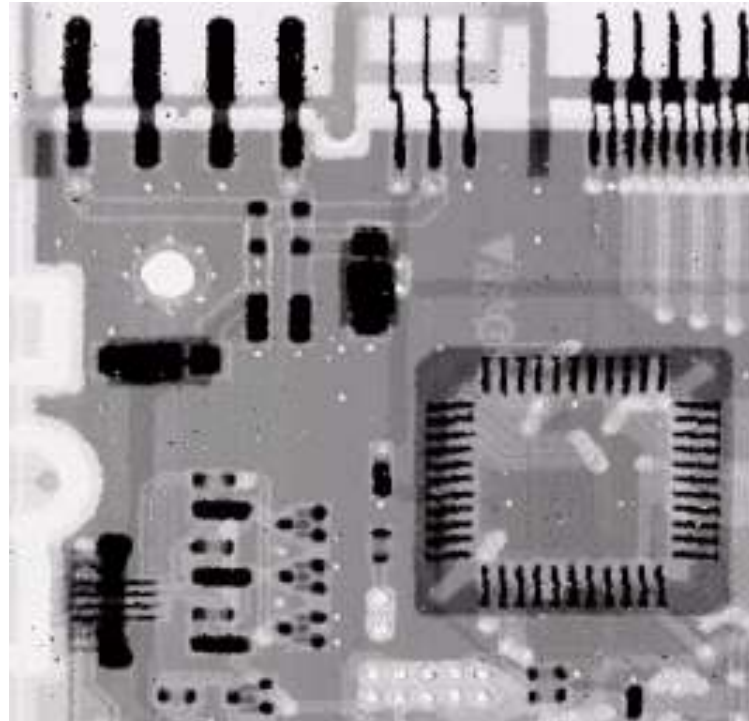
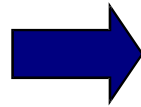
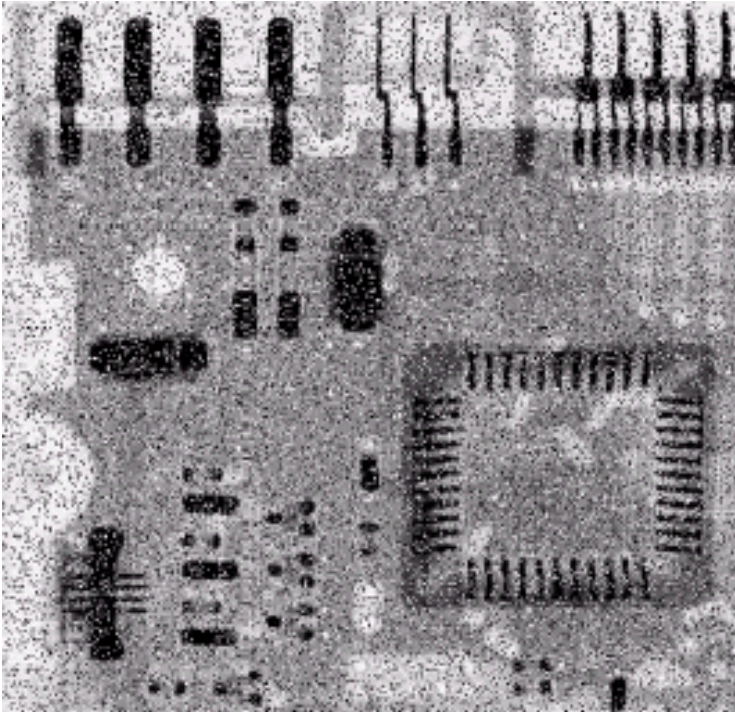


Image Enhancement Examples (cont...)

Images taken from Gonzalez & Woods, Digital Image Processing (2002)



Spatial & Frequency Domains

There are two broad categories of image enhancement techniques

- Spatial domain techniques
 - Direct manipulation of image pixels (intensity values)
- Frequency domain techniques
 - Manipulation of Fourier transform or wavelet transform of an image

First spatial domain techniques

Later: frequency domain techniques

What is a Filter?

- Capabilities of point operations are limited
- **Filters:** combine pixel's value + values of neighbors
- **E.g blurring:** Compute average intensity of block of pixels



- Combining multiple pixels needed for certain operations:
 - Blurring, Smoothing
 - Sharpening

What Point Operations Can't Do

- Example: sharpening



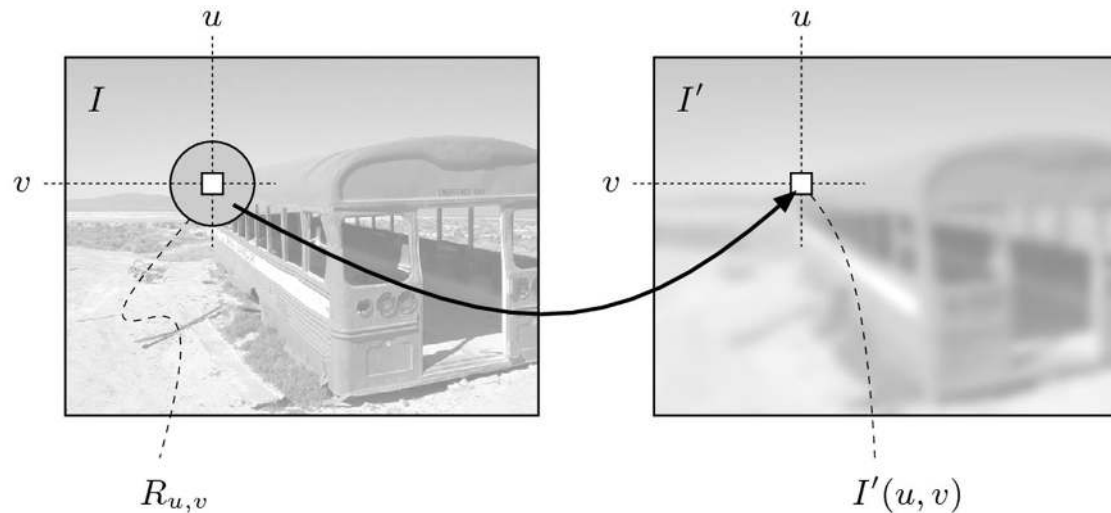
What Point Operations Can't Do

- Other cool artistic patterns by combining pixels

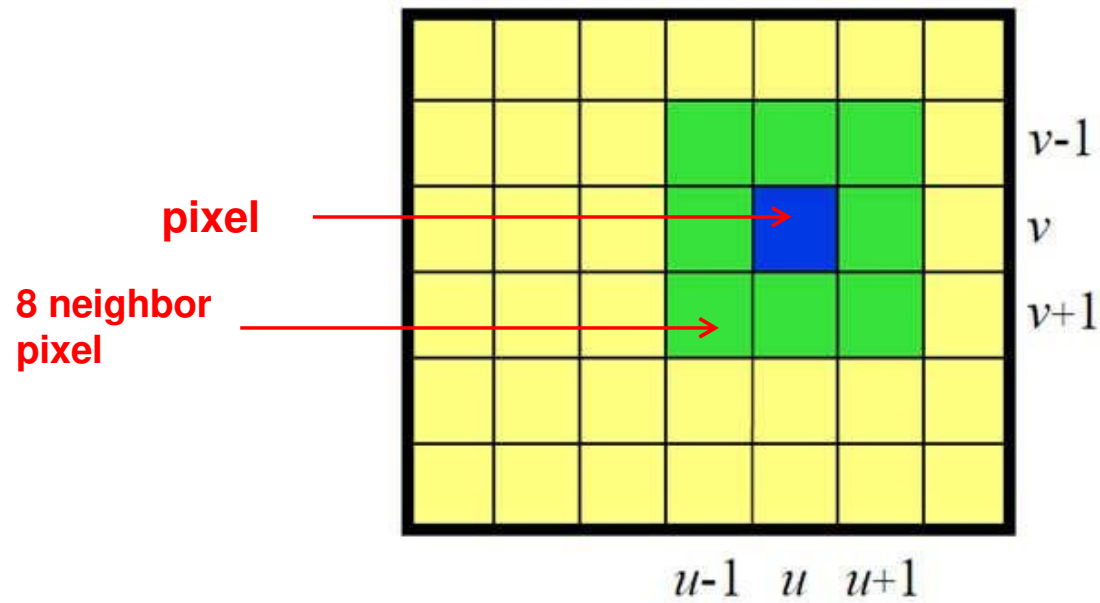


Definition: Spatial Filter

- An image operation that combines each pixel's intensity $I(u, v)$ with that of neighboring pixels
- **E.g:** average/weighted average of group of pixels



Example: Average (Mean) of 3x3 Neighborhood

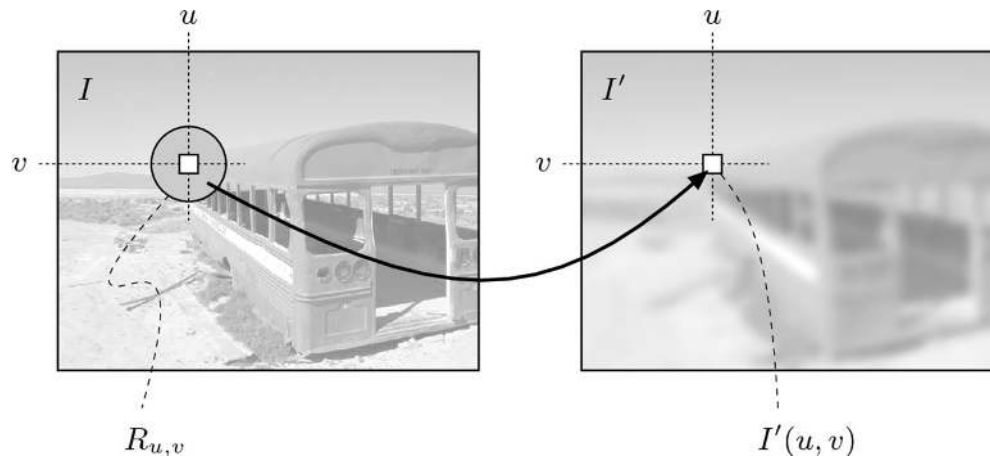
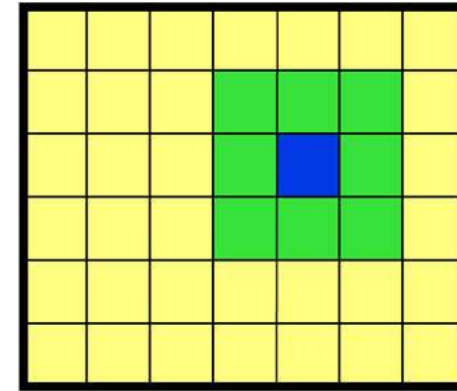


Blurring: Replace each pixel with AVERAGE Intensity of pixel + neighbors

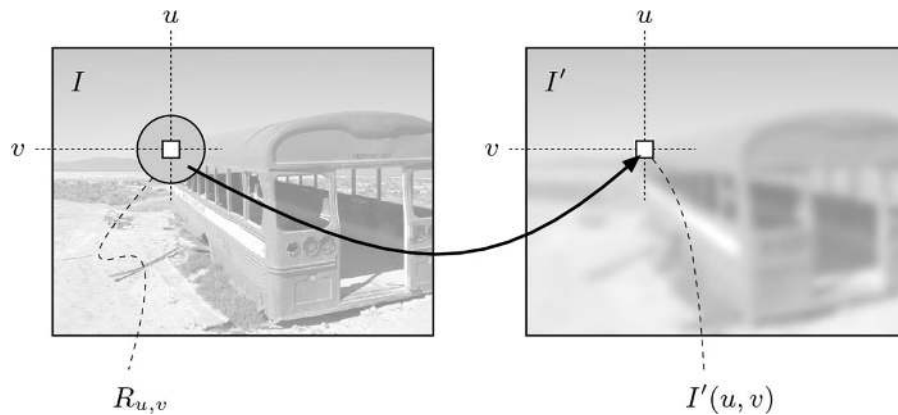
Smoothing an Image by Averaging

- Replace each pixel by average of pixel + neighbors
- For 3x3 neighborhood:

$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$



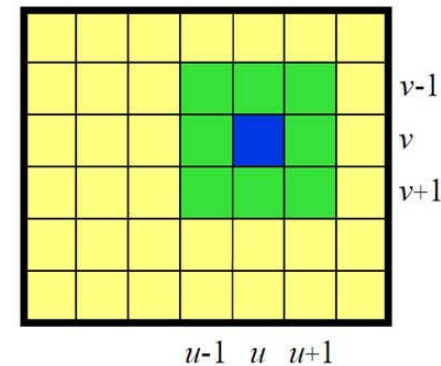
Smoothing an Image by Averaging



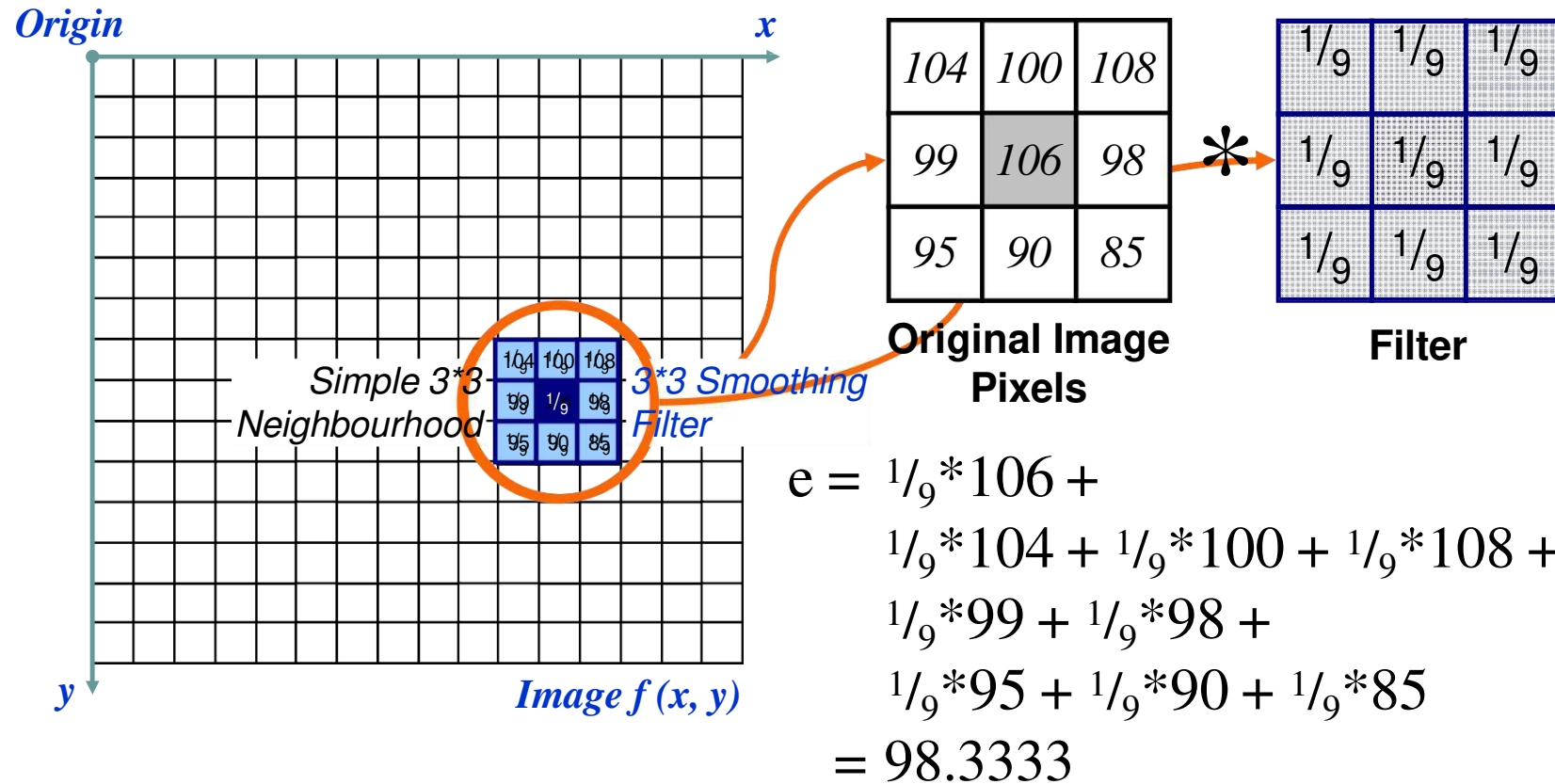
$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot \sum_{j=-1}^1 \sum_{i=-1}^1 I(u+i, v+j)$$

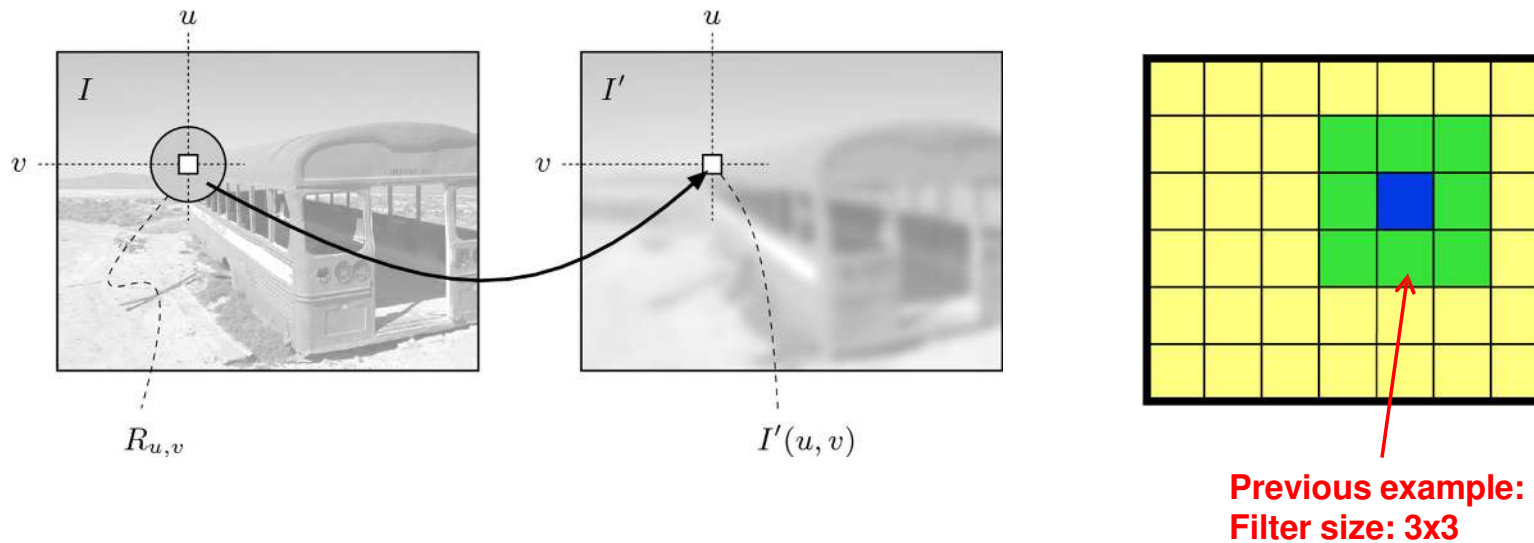


Example: Smoothing Spatial Filtering



The above is repeated for every pixel in the original image to generate the smoothed image

Smoothing an Image by Averaging



- Many possible filter parameters (size, weights, function, etc)
- **Filter size (size of neighborhood):** 3x3, 5x5, 7x7, ..., 21x21, ...
- **Filter shape:** not necessarily square. Can be rectangle, circle, etc
- **Filter weights:** May apply unequal weighting to different pixels
- **Filters function:** can be linear (a weighted summation) or nonlinear

The Filter Matrix

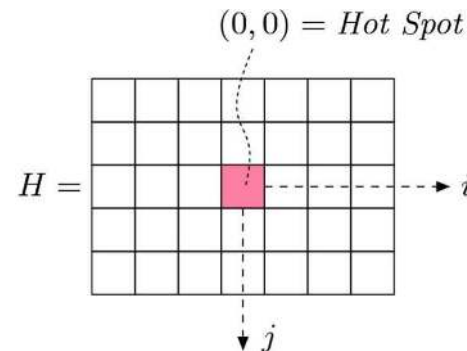
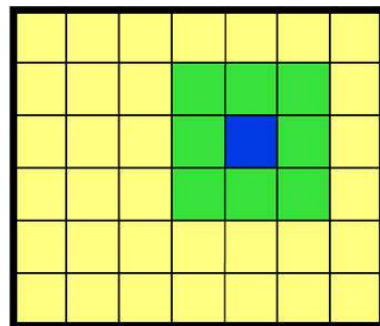
$$I'(u, v) \leftarrow \frac{p_0 + p_1 + p_2 + p_3 + p_4 + p_5 + p_6 + p_7 + p_8}{9}$$

$$I'(u, v) \leftarrow \frac{1}{9} \cdot [I(u-1, v-1) + I(u, v-1) + I(u+1, v-1) + \\ I(u-1, v) + I(u, v) + I(u+1, v) + \\ I(u-1, v+1) + I(u, v+1) + I(u+1, v+1)]$$

$$H(i, j) = \begin{bmatrix} 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \\ 1/9 & 1/9 & 1/9 \end{bmatrix} = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Filter operation can be
expressed as a matrix
Example: averaging filter

Filter matrix also called
filter mask $H(i, j)$



Example: What does this Filter Do?



0	0	0
0	1	0
0	0	0



Identity function (leaves image alone)

What Does this Filter Do?


$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1



Mean (averages neighborhood)

Mean Filters: Effect of Filter Size



Original



7×7



15×15



41×41

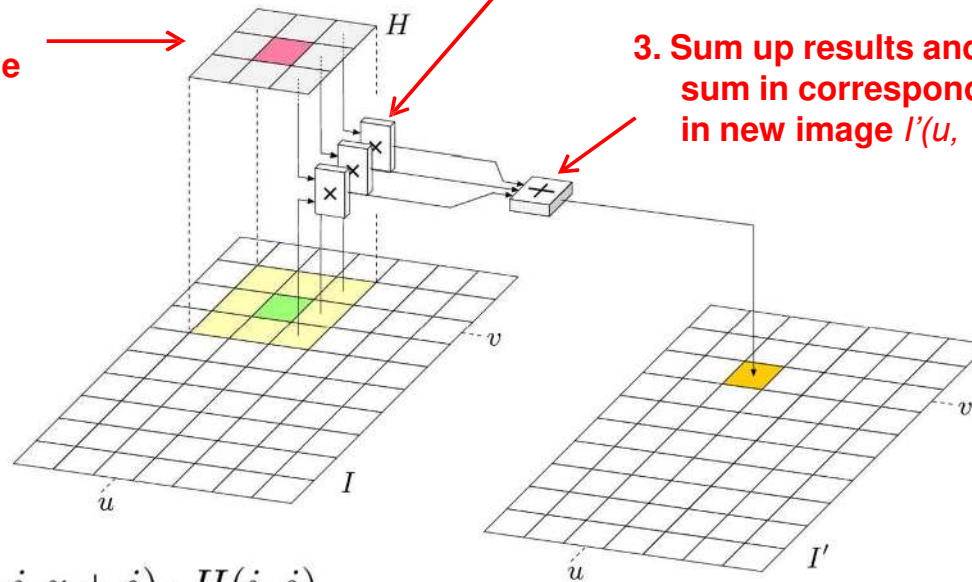
Applying Linear Filters: Convolution

For each image position $I(u,v)$:

1. Move filter matrix H over image such that $H(0,0)$ coincides with current image position (u,v)

2. Multiply all filter coefficients $H(i,j)$ with corresponding pixel $I(u+i, v+j)$

3. Sum up results and store sum in corresponding position in new image $I'(u, v)$



Stated formally:

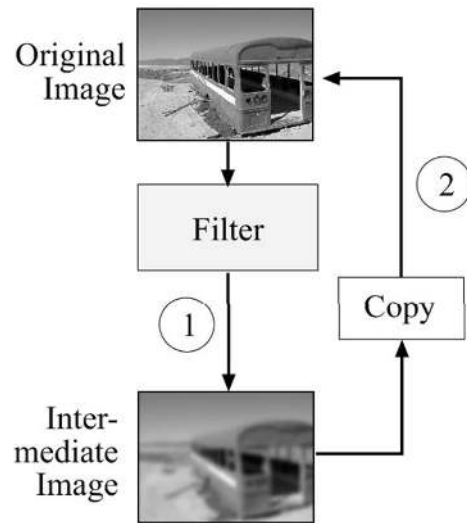
$$I'(u, v) \leftarrow \sum_{(i,j) \in R_H} I(u+i, v+j) \cdot H(i, j)$$

R_H is set of all pixels covered by filter.
For 3x3 filter, this is:

$$I'(u, v) \leftarrow \sum_{i=-1}^{i=1} \sum_{j=-1}^{j=1} I(u+i, v+j) \cdot H(i, j)$$

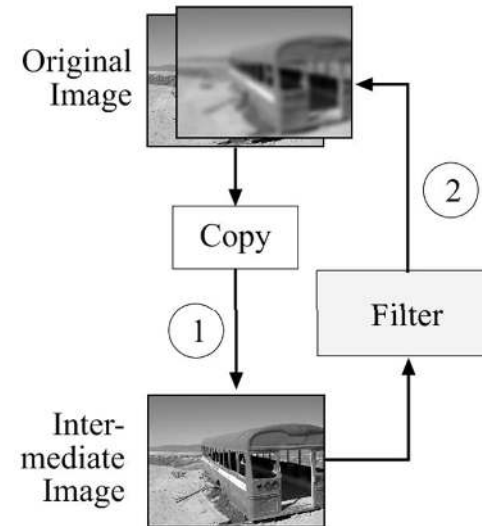
Computing Filter Operation

- Filter matrix H moves over each pixel in original image I to compute corresponding pixel in new image I'
- Cannot overwrite new pixel value in original image I Why?



Version A

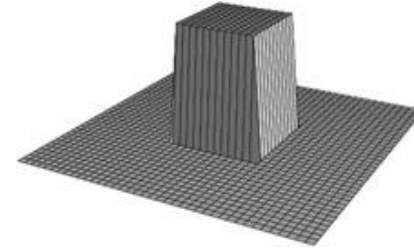
Store results I' in intermediate image, then copy back to replace I



Version B

Copy original image I to intermediate image, use it as source, then store results I' to replace original image

Simple 3x3 Averaging Filter (“Box” Filter)



```
1 import ij.*;
2 import ij.plugin.filter.PlugInFilter;
3 import ij.process.*;
4
5 public class Filter_Average3x3 implements PlugInFilter {
6     ...
7     public void run(ImageProcessor orig) {
8         int w = orig.getWidth();
9         int h = orig.getHeight();
10        ImageProcessor copy = orig.duplicate();
11
12        for (int v = 1; v <= h-2; v++) {
13            for (int u = 1; u <= w-2; u++) {
14                //compute filter result for position (u,v)
15                int sum = 0;
16                for (int j = -1; j <= 1; j++) {
17                    for (int i = -1; i <= 1; i++) {
18                        int p = copy.getPixel(u+i, v+j);
19                        sum = sum + p;
20                    }
21                }
22                int q = (int) Math.round(sum/9.0);
23                orig.putPixel(u, v, q);
24            }
25        }
26    }
27 } // end of class Filter_Average3x3
```

No explicit filter matrix since all coefficients are the same ($1/9$)

No clamping required

Make copy of original image to use as source

Loop over all pixels in image

Filter computation by adding current pixel's neighbors

Store result back in original image

Weighted Smoothing Filters

- More effective smoothing filters can be generated by allowing different pixels in the neighbourhood different weights in the averaging function
 - Pixels closer to central pixel more important
 - Often referred to as a *weighted averaging*

$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$
$\frac{2}{16}$	$\frac{4}{16}$	$\frac{2}{16}$
$\frac{1}{16}$	$\frac{2}{16}$	$\frac{1}{16}$

Weighted
averaging filter

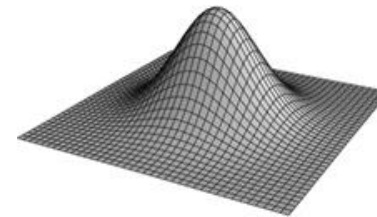
Another Smoothing Filter

```
1 public void run(ImageProcessor orig) {
2     int w = orig.getWidth();
3     int h = orig.getHeight();
4     // 3 × 3 filter matrix
5     double[][] filter = {
6         {0.075, 0.125, 0.075},
7         {0.125, 0.200, 0.125},
8         {0.075, 0.125, 0.075}
9     };
10    ImageProcessor copy = orig.duplicate();
11
12    for (int v = 1; v <= h-2; v++) {
13        for (int u = 1; u <= w-2; u++) {
14            // compute filter result for position (u,v)
15            double sum = 0;
16            for (int j = -1; j <= 1; j++) {
17                for (int i = -1; i <= 1; i++) {
18                    int p = copy.getPixel(u+i, v+j);
19                    // get the corresponding filter coefficient:
20                    double c = filter[j+1][i+1];
21                    sum = sum + c * p;
22                }
23            }
24            int q = (int) Math.round(sum);
25            orig.putPixel(u, v, q);
26        }
27    }
28 }
```

Use real filter matrix with coefficients
Apply bell-shaped function $H(i,j)$

$$H(i,j) = \begin{bmatrix} 0.075 & 0.125 & 0.075 \\ 0.125 & \mathbf{0.2} & 0.125 \\ 0.075 & 0.125 & 0.075 \end{bmatrix}$$

Bell-shaped function $H(i,j)$?
- More weight applied to center



Apply filter

Store result back in original

Integer Coefficients

- Instead of floating point coefficients, more efficient, simpler to use:

scaling factor + integer coefficients

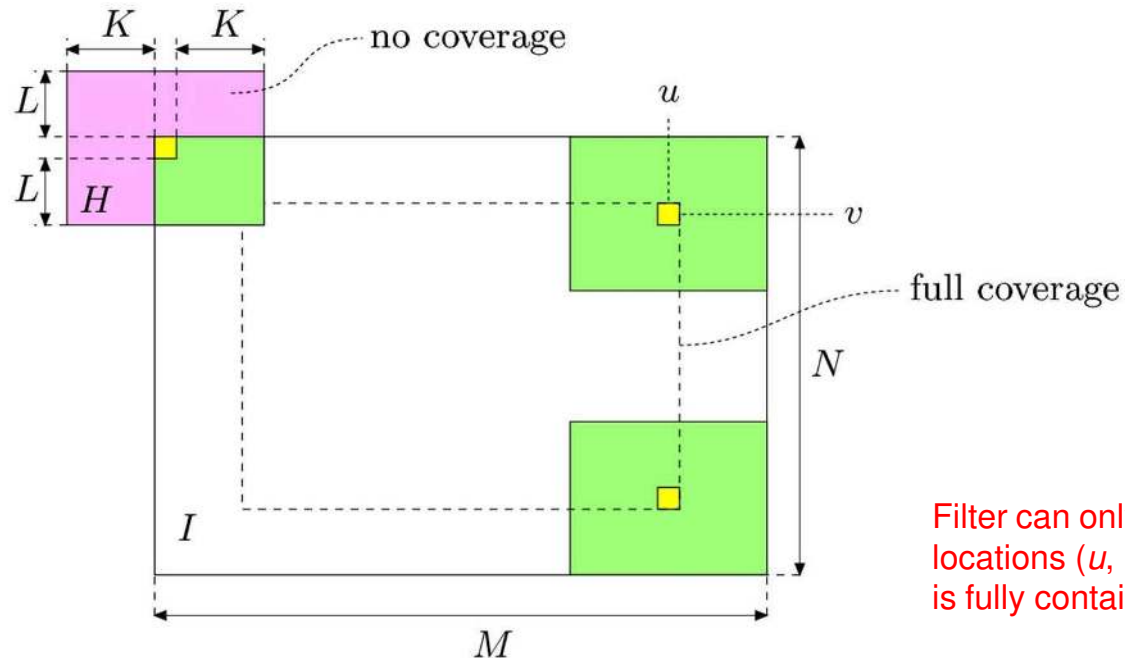
$$H(i, j) = \begin{bmatrix} 0.075 & 0.125 & 0.075 \\ 0.125 & \underline{0.200} & 0.125 \\ 0.075 & 0.125 & 0.075 \end{bmatrix} = \frac{1}{40} \begin{bmatrix} 3 & 5 & 3 \\ 5 & \underline{8} & 5 \\ 3 & 5 & 3 \end{bmatrix}$$

$$H(i, j) = s \cdot H'(i, j)$$

Computation Range

- For a filter of size $(2K+1) \times (2L+1)$, if image size is $M \times N$, filter is computed over the range:

$$K \leq u' \leq (M-K-1) \quad \text{and} \quad L \leq v' \leq (N-L-1)$$



Filter can only be applied at image locations (u, v) where filter matrix H is fully contained in the image

Filter of Arbitrary Size

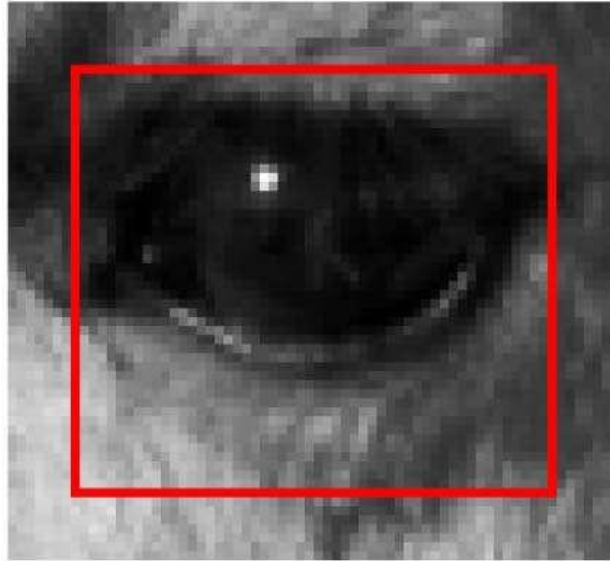
```
1 public void run(ImageProcessor orig) {
2     int M = orig.getWidth();
3     int N = orig.getHeight();
4
5     // filter matrix of size  $(2K + 1) \times (2L + 1)$ 
6     int[][] filter = {
7         {0,0,1,1,1,0,0},
8         {0,1,1,1,1,1,0},
9         {1,1,1,1,1,1,1},
10        {0,1,1,1,1,1,0},
11        {0,0,1,1,1,0,0}
12    };
13    double s = 1.0/23; // sum of filter coefficients is 23
14
15    int K = filter[0].length/2;
16    int L = filter.length/2;
17
18    ImageProcessor copy = orig.duplicate();
19
20    for (int v = L; v <= N-L-1; v++) {
21        for (int u = K; u <= M-K-1; u++) {
22            // compute filter result for position (u,v)
23            int sum = 0;
24            for (int j = -L; j <= L; j++) {
25                for (int i = -K; i <= K; i++) {
26                    int p = copy.getPixel(u+i, v+j);
27                    int c = filter[j+L][i+K];
28                    sum = sum + c * p;
29                }
30            }
31            int q = (int) Math.round(s * sum);
32            if (q < 0) q = 0;
33            if (q > 255) q = 255;
34            orig.putPixel(u, v, q);
35        }
36    }
37 }
```

← Declare filter

← Clamp result as a precaution

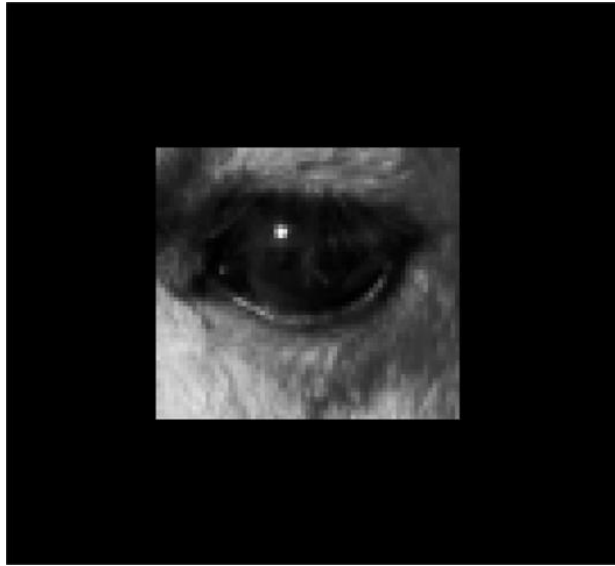
What to do at image boundaries?

a) Crop



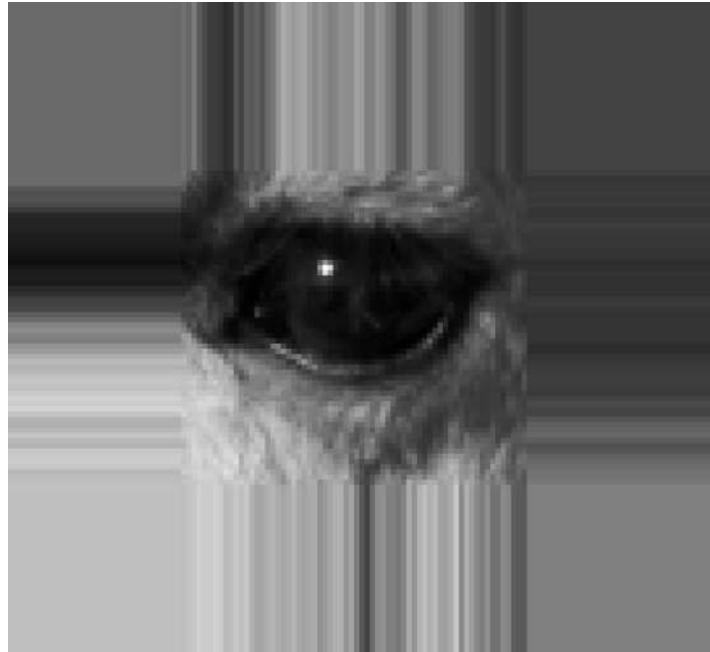
What to do at image boundaries?

- a) Crop
- b) Pad



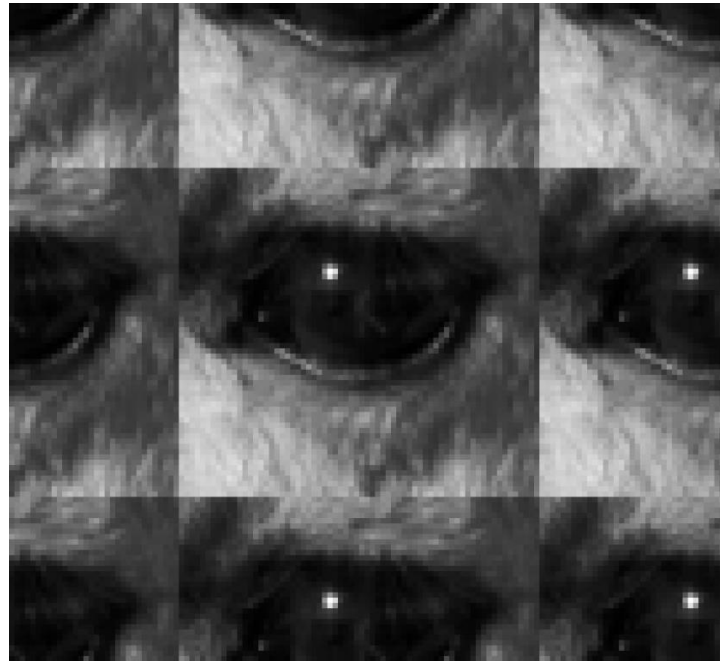
What to do at image boundaries?

- a) Crop
- b) Pad
- c) Extend



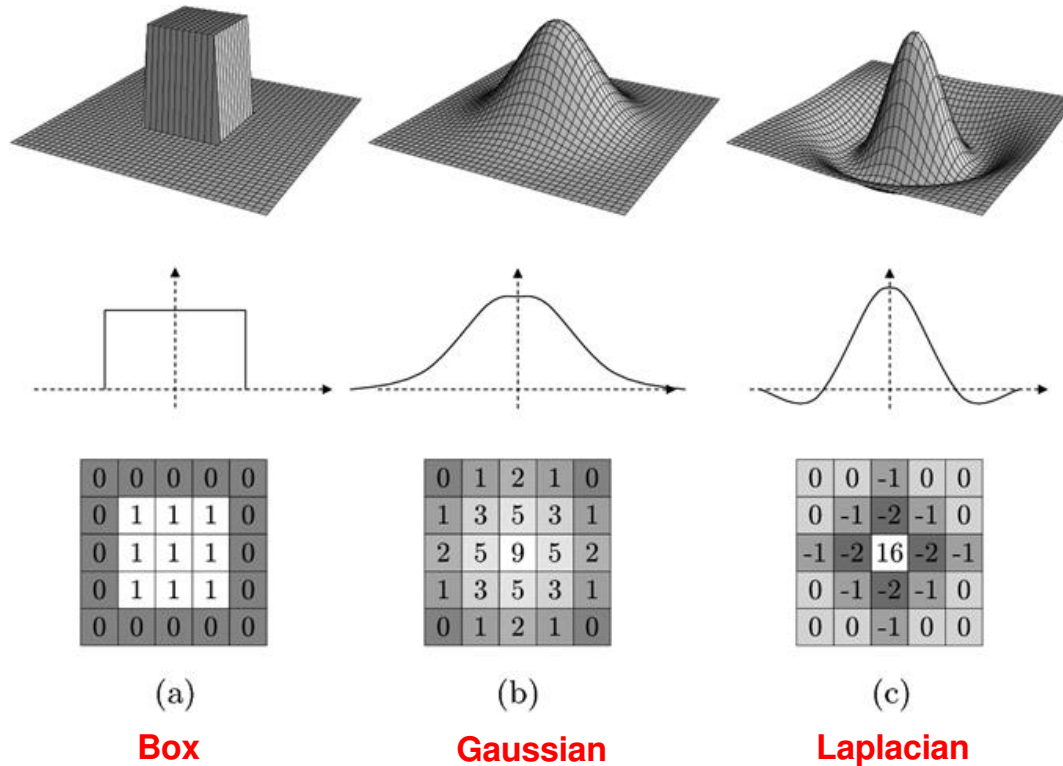
What to do at image boundaries?

- a) Crop
- b) Pad
- c) Extend
- d) Wrap



Linear Filters: Smoothing Filters

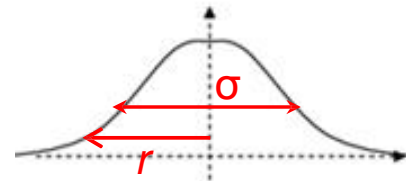
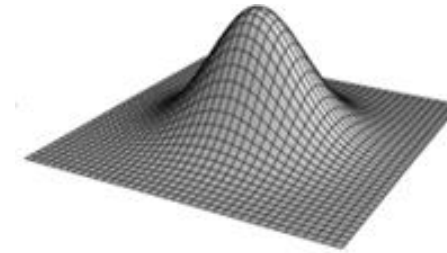
- 2 main classes of linear filters:
 - **Smoothing**: +ve coefficients (weighted average). E.g box, gaussian
 - **Difference** filters: +ve and -ve weights. E.g. Laplacian



Gaussian Filter

$$G_{\sigma}(r) = e^{-\frac{r^2}{2\sigma^2}} \quad \text{or} \quad G_{\sigma}(x, y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- where
 - σ is width (standard deviation)
 - r is distance from center



0	1	2	1	0
1	3	5	3	1
2	5	9	5	2
1	3	5	3	1
0	1	2	1	0

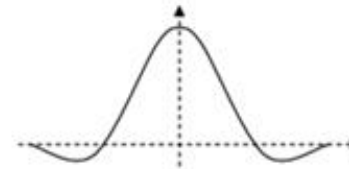
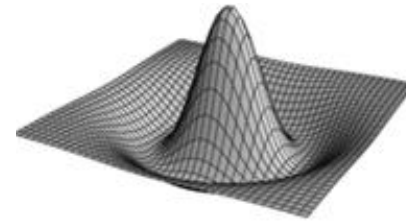
**Gaussian
filter**

Difference Filters

- **Coefficients:** some +ve, some negative
- Example: Laplacian filter
- Computation is difference

$$\sum (+ve \text{ coefficients}) - \sum (-ve \text{ coefficients})$$

$$I'(u, v) = \sum_{(i,j) \in R_H^+} I(u+i, v+j) \cdot |H(i, j)| \\ - \sum_{(i,j) \in R_H^-} I(u+i, v+j) \cdot |H(i, j)|$$



0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0

**Laplacian
filter**

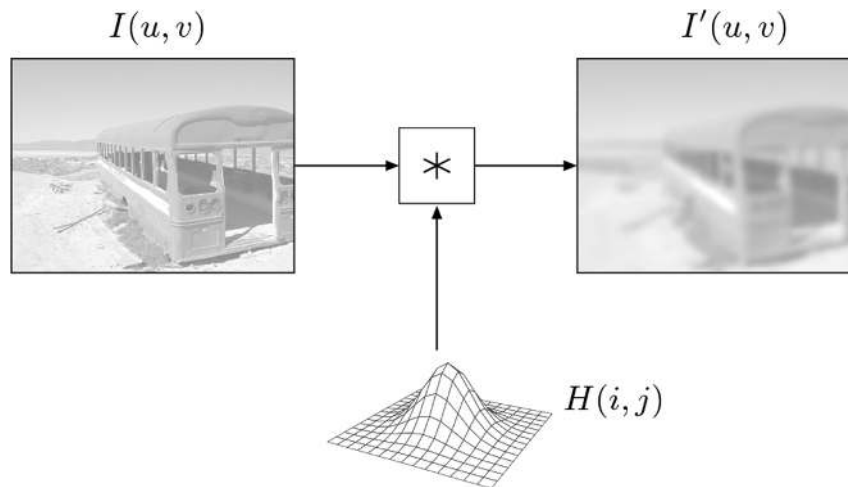
Mathematical Properties of Convolution

- Applying a filter as described called *linear convolution*
- For discrete 2D signal, convolution defined as:

$$I'(u, v) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} I(u-i, v-j) \cdot H(i, j)$$

Formal definition:
Sum to $\pm \infty$

$$I' = I * H$$



Properties of Convolution

- Commutativity

$$I * H = H * I$$

Same result if we convolve
image with filter or vice versa

- Linearity

$$(s \cdot I) * H = I * (s \cdot H) = s \cdot (I * H)$$

If image multiplied by scalar
Result multiplied by same scalar

$$(I_1 + I_2) * H = (I_1 * H) + (I_2 * H)$$

(notice)

$$(b + I) * H \neq b + (I * H)$$

If 2 images added and convolve
result with a kernel H ,
Same result if each image
is convolved individually + added

- Associativity

$$A * (B * C) = (A * B) * C$$

Order of filter application irrelevant
Any order, same result

Separability in x and y

- Sometimes we can separate a kernel into “vertical” and “horizontal” components
- Consider the kernels

$$H_x = [1 \ 1 \ 1 \ 1 \ 1], \quad \text{and} \quad H_y = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Then

$$H = H_x * H_y = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Complexity of x/y Separable Kernels

- What is the number of operations for 3×5 kernel H

Ans: $15wh$

$$H = H_x * H_y = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

- What is the number of operations for H_x followed by H_y ?

Ans: $3wh + 5wh = 8wh$

$$H_x = [1 \ 1 \ 1 \ 1 \ 1], \quad \text{and} \quad H_y = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Complexity of x/y Separable Kernels

- What is the number of operations for 3×5 kernel H

Ans: $15wh$

- What is the number of operations for H_x followed by H_y ?

Ans: $3wh + 5wh = 8wh$

- What about $M \times M$ kernel?

$O(M^2)$ – no separability (M^2wh operations, **grows quadratically!**)

$O(M^2)$ – with separability ($2Mwh$ operations, **grows linearly!**)

Gaussian Kernel

- 1D

$$g_{\sigma}(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right)$$

- 2D

$$G_{\sigma}(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Separability of 2D Gaussian

- 2D gaussian is just product of 1D gaussians:

$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \\ &= \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{x^2}{2\sigma^2}\right) \cdot \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{y^2}{2\sigma^2}\right) \\ &= g_{\sigma}(x) \cdot g_{\sigma}(y) \end{aligned}$$

Separable!



Separability of 2D Gaussian

- Consequently, convolution with a gaussian is separable

$$I * G = I * G_x * G_y$$

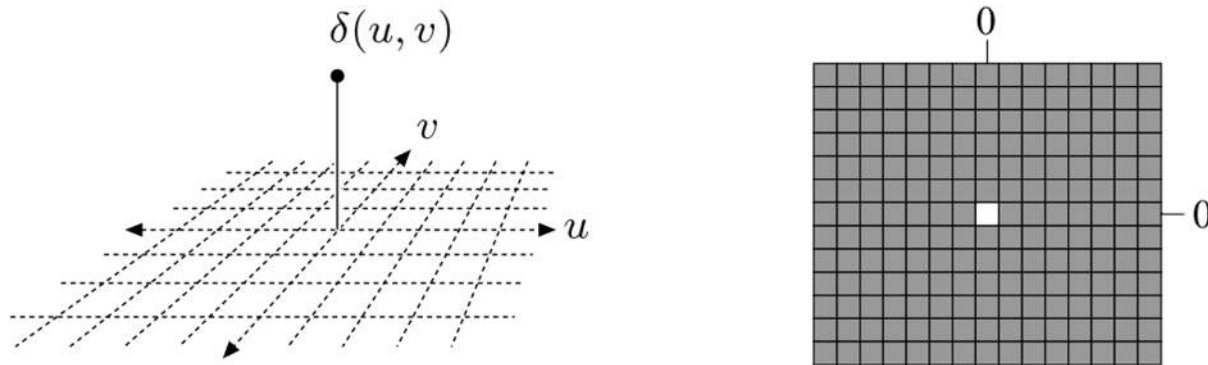
- Where G is the 2D discrete gaussian kernel;
- G_x is “horizontal” and G_y is “vertical” 1D discrete Gaussian kernels

Impulse (or Dirac) Function

- In discrete 2D case, impulse function defined as:

$$\delta(u, v) = \begin{cases} 1 & \text{for } u = v = 0 \\ 0 & \text{otherwise.} \end{cases}$$

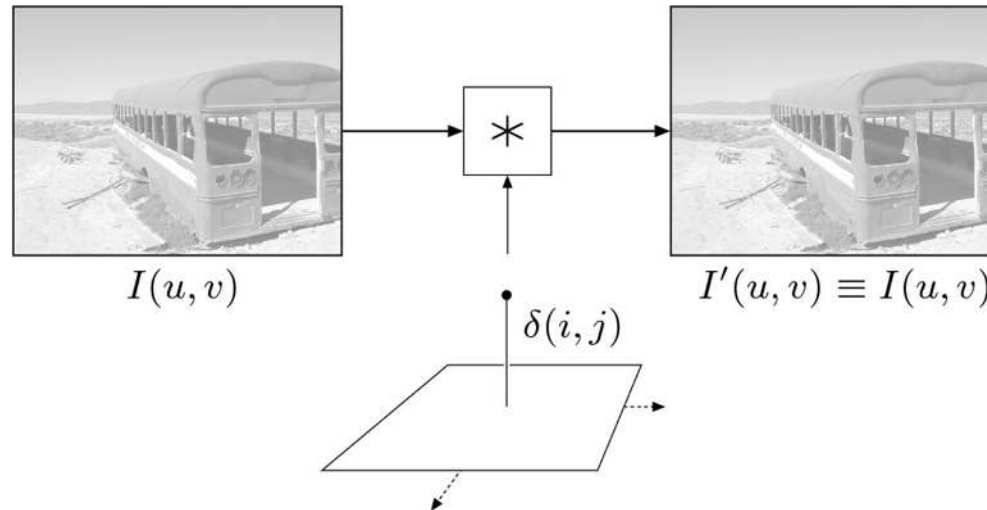
- Impulse function on image?
 - A white pixel at origin, on black background



Impulse (or Dirac) Function

- Impulse function neutral under convolution (no effect)
- Convolving an image using impulse function as filter = image

$$I * \delta = I$$

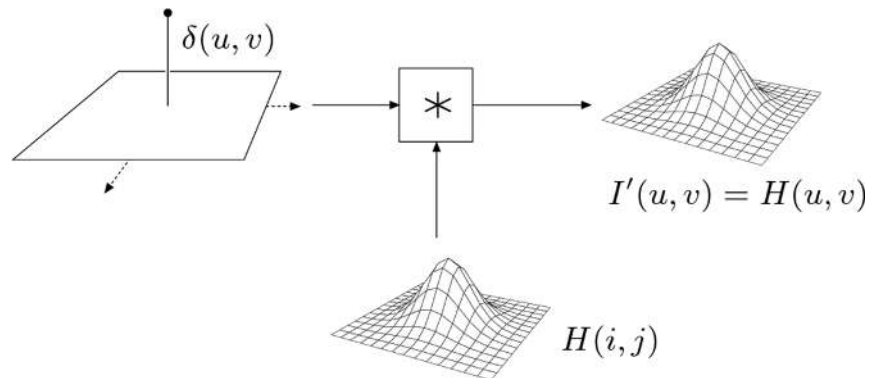


Impulse (or Dirac) Function

- Reverse case? Apply filter H to impulse function
- Using fact that convolution is commutative

$$H * \delta = \delta * H = H$$

- Result is the filter H



Noise



- While taking picture (during capture), noise may occur
- Noise? Errors, degradations in pixel values
- Examples of causes:
 - Focus blurring
 - Blurring due to camera motion
- Additive model for noise:
$$H * I + \text{Noise}$$
- Removing noise called **Image Restoration**
- Image restoration can be done in:
 - Spatial domain, or
 - Frequency domain

Types of Noise

- Type of noise determines best types of filters for removing it!!
- **Salt and pepper noise:** Randomly scattered black + white pixels
- Also called **impulse noise, shot noise or binary noise**
- Caused by sudden sharp disturbance



(a) Original image



(b) With added salt & pepper noise

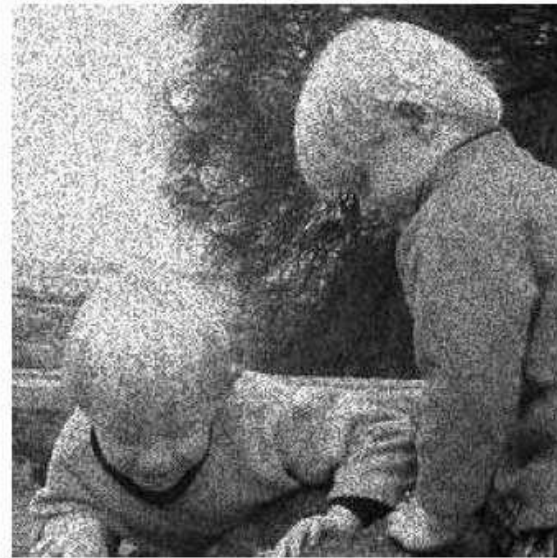
*Courtesy
Allasdair McAndrews*

Types of

- **Noise** Gaussian Noise: idealized form of white noise *added to* image, normally distributed $I + \text{Noise}$
- **Noise** Speckle Noise: pixel values *multiplied by* random noise $I(1 + \text{Noise})$



(a) Gaussian noise



(b) Speckle noise

*Courtesy
Allasdair McAndrews*

Types of Noise

- **Periodic Noise:** caused by disturbances of a periodic nature
- Salt and pepper, gaussian and speckle noise can be cleaned using spatial filters
- Periodic noise can be cleaned using frequency domain filtering (later)

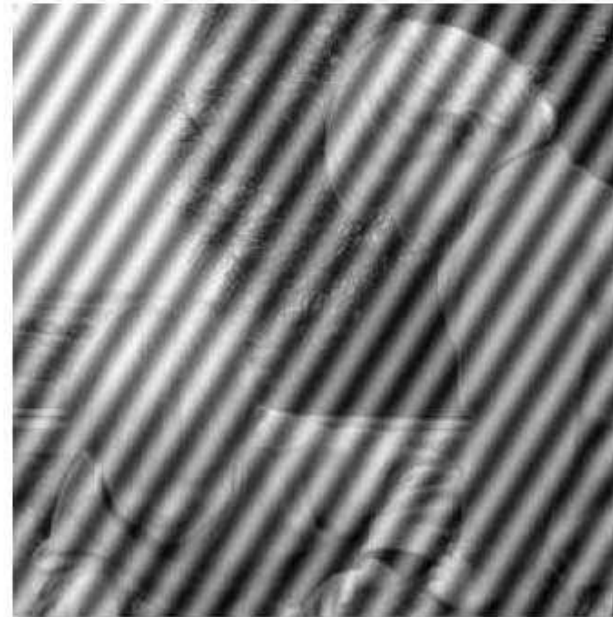
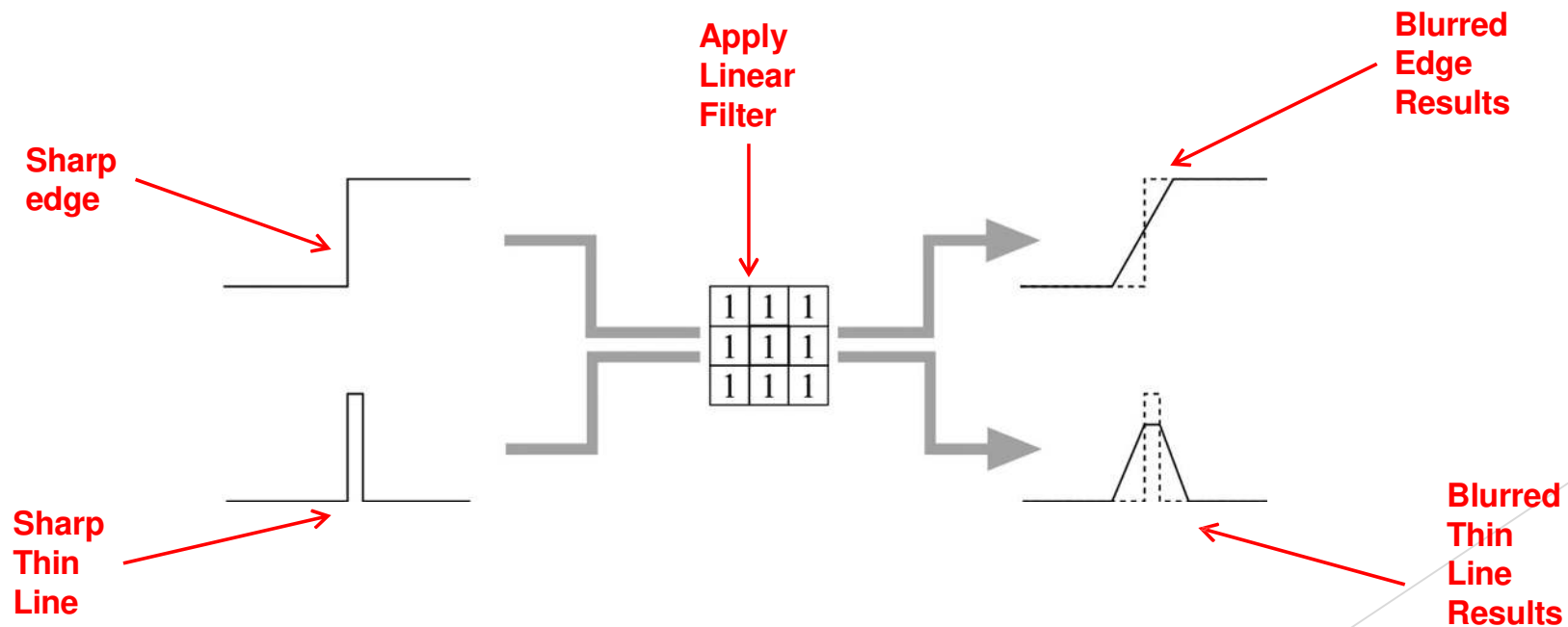


Figure 5.3: The twins image corrupted by periodic noise

*Courtesy
Allasdair McAndrews*

Non-Linear Filters

- Linear filters blurs all image structures points, edges and lines, reduction of image quality (**bad!**)
- Linear filters thus not used a lot for removing noise



Using Linear Filter to Remove

• **Noise?** Example: Using linear filter to clean salt and pepper noise just causes smearing (not clean removal)

- Try non-linear filters?

*Courtesy
Allasdair McAndrews*



(a) 3×3 averaging



(b) 7×7 averaging

Non-Linear

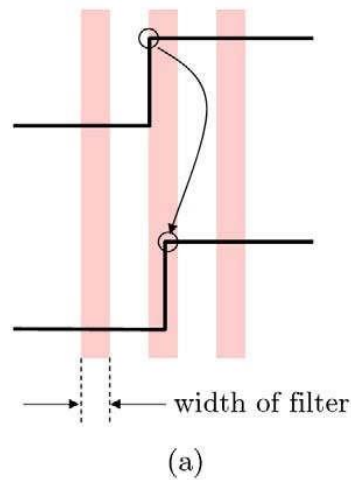
- Filters
- Pixels in filter range combined by some non-linear function
- Simplest examples of nonlinear filters: Min and Max filters

$$I'(u, v) \leftarrow \min \{ I(u+i, v+j) \mid (i, j) \in R \}$$

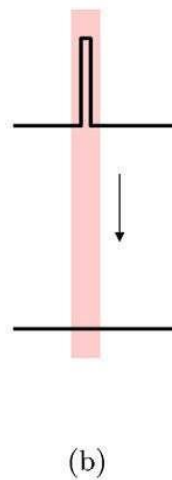
$$I'(u, v) \leftarrow \max \{ I(u+i, v+j) \mid (i, j) \in R \}$$

Before
filtering

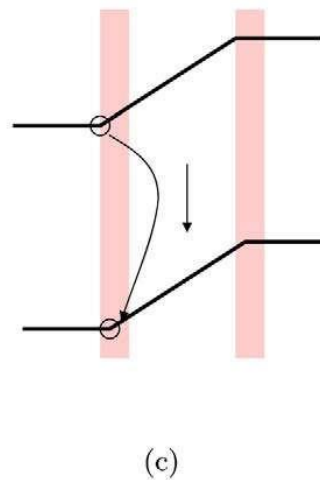
After
filtering



Step Edge
(shifted to right)



Narrow
Pulse
(removed)



Linear Ramp
(shifted to right)

Effect of
Minimum
filter

Non-linear Filters



(a)

**Original Image with
Salt-and-pepper noise**



(b)

**Minimum filter removes
bright spots (maxima) and
widens dark image structures**



(c)

**Maximum filter (opposite effect):
Removes dark spots (minima) and
widens bright image structures**

Median Filter

- Much better at removing noise and keeping the structures

$$I'(u, v) \leftarrow \text{median} \{ I(u+i, v+j) \mid (i, j) \in R \}$$

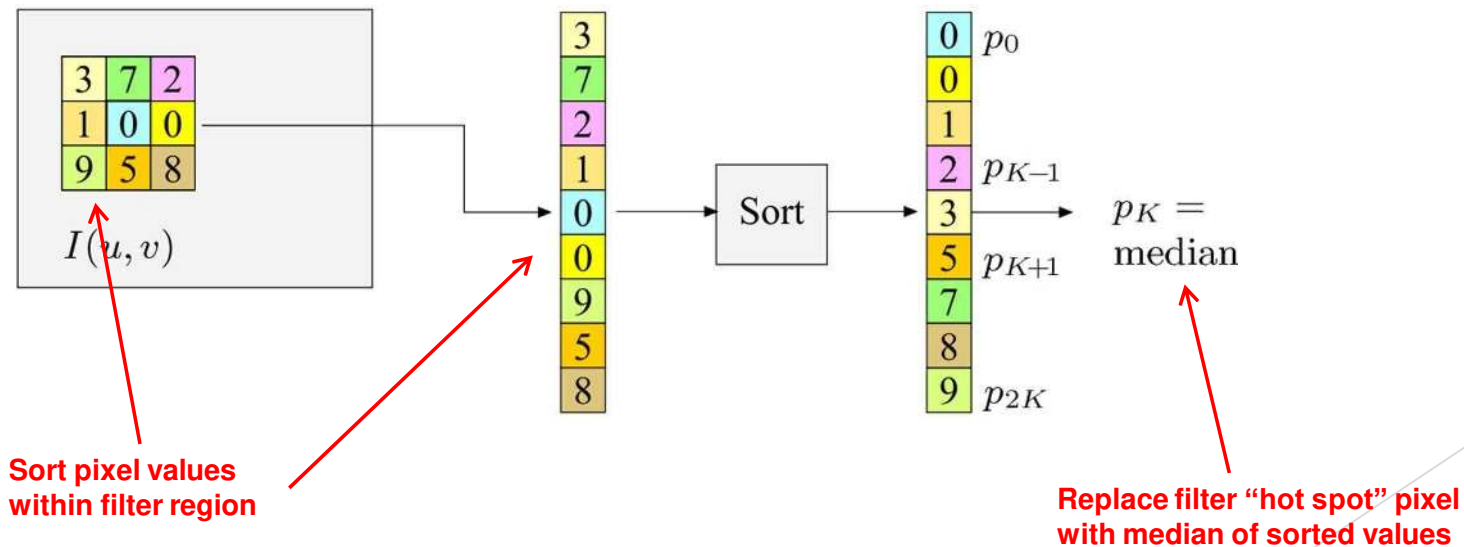
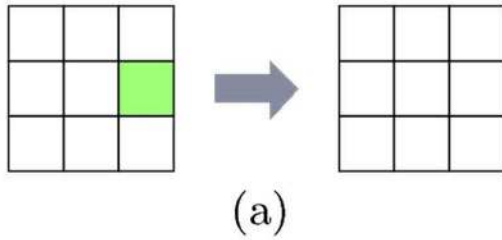
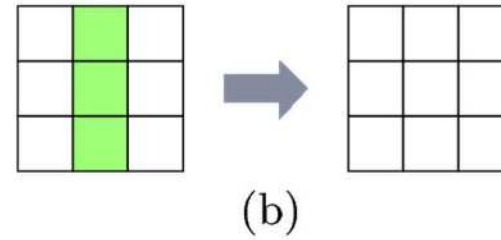


Illustration: Effects of Median Filter

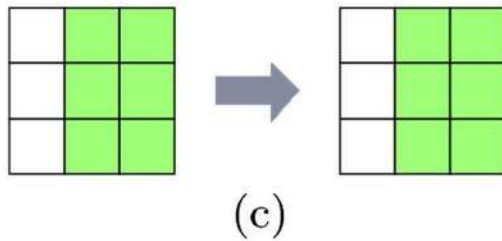
Isolated pixels
are eliminated



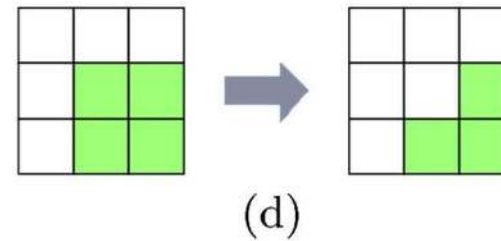
Thin lines
are eliminated



A step edge is
unchanged



A corner is
rounded off



Effects of Median Filter



(a)

**Original Image with
Salt-and-pepper noise**



(b)

**Linear filter removes some of
the noise, but not completely.
Smears noise**



(c)

**Median filter salt-and-pepper noise
and keeps image structures largely
intact. But also creates small spots
of flat intensity, that affect sharpness**

Median Filter ImageJ Plugin

```
1 import ij.*;
2 import ij.plugin.filter.PlugInFilter;
3 import ij.process.*;
4 import java.util.Arrays;
5
6 public class Filter_Median3x3 implements PlugInFilter {
7     final int K = 4; // filter size
8
9     public void run(ImageProcessor orig) {
10         int w = orig.getWidth();
11         int h = orig.getHeight();
12         ImageProcessor copy = orig.duplicate();
13
14         // vector to hold pixels from 3x3 neighborhood
15         int[] P = new int[2*K+1];
16
17         for (int v = 1; v <= h-2; v++) {
18             for (int u = 1; u <= w-2; u++) {
19                 // fill the pixel vector P for filter position u, v
20                 int k = 0;
21                 for (int j = -1; j <= 1; j++) {
22                     for (int i = -1; i <= 1; i++) {
23                         P[k] = copy.getPixel(u+i, v+j);
24                         k++;
25                     }
26                 }
27                 // sort pixel vector and take the center element
28                 Arrays.sort(P);
29                 orig.putPixel(u, v, P[K]);
30             }
31         }
32     }
33 } // end of class Filter_Median3x3
```

Get Image width + height,
and Make copy of image

Array to store pixels to be filtered. Good
data structure in which to find median

Copy pixels within filter
region into array

Sort pixels within filter using
java utility Arrays.sort()

Middle (k) element of sorted array
assumed to be middle. Return as median

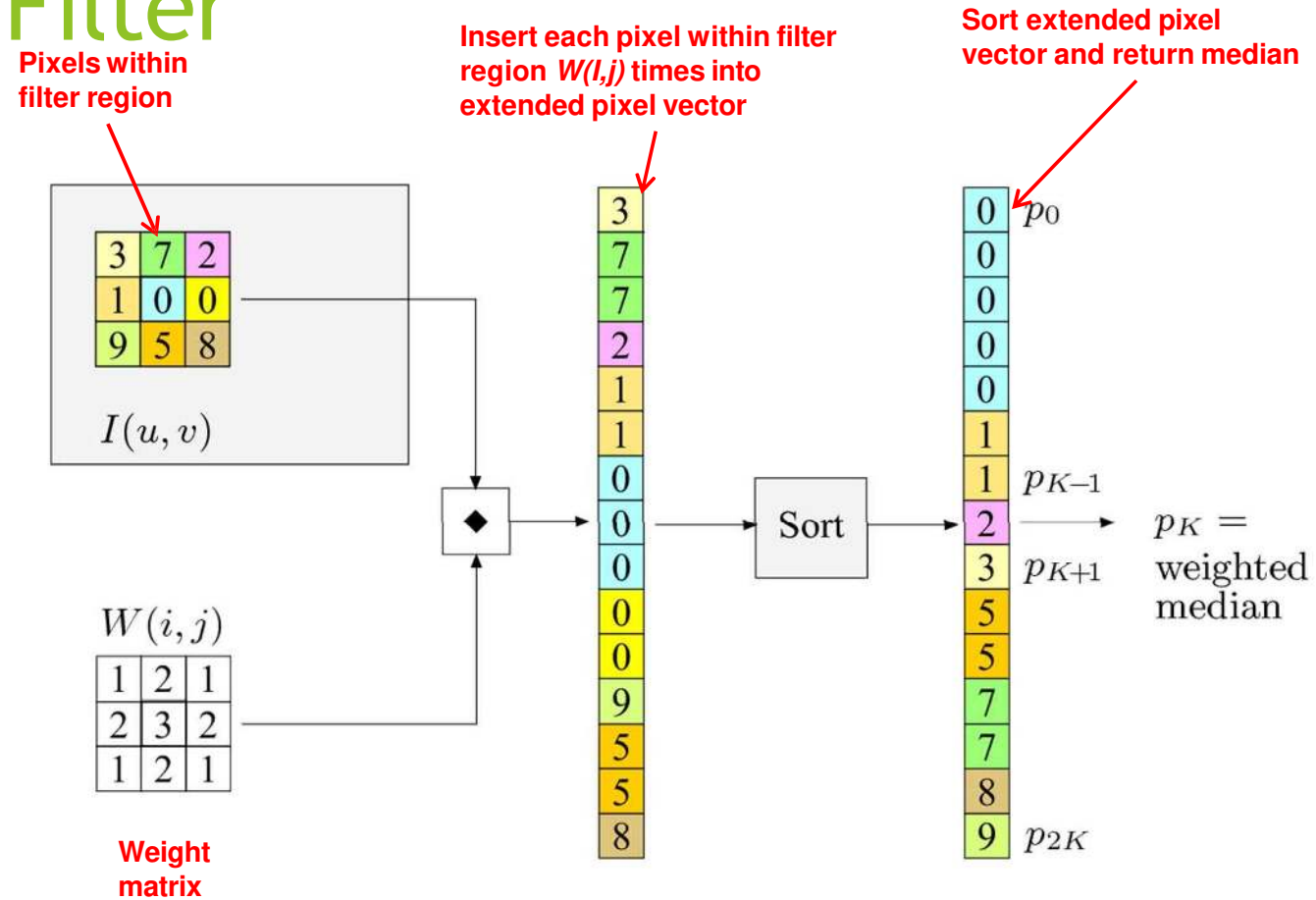
Weighted Median Filter

- Color assigned by median filter determined by colors of “the majority” of pixels within the filter region
- Considered robust since single high or low value cannot influence result (unlike linear average)
- Median filter assigns weights (number of “votes”) to filter positions

$$W(i, j) = \begin{bmatrix} 1 & 2 & 1 \\ 2 & \mathbf{3} & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- To compute result, each pixel value within filter region is inserted $W(i, j)$ times to create **extended pixel vector**
- Extended pixel vector then sorted and median returned

Weighted Median Filter



Note: assigning weight to center pixel larger than sum of all other pixel weights inhibits any filter effect (center pixel always carries majority)!!

Weighted Median Filter

- More formally, **extended pixel vector** defined as

$$Q = (p_0, \dots, p_{L-1}) \quad \text{of length} \quad L = \sum_{(i,j) \in R} W(i,j)$$

- For example, following weight matrix yields extended pixel vector of length 15 (sum of weights)

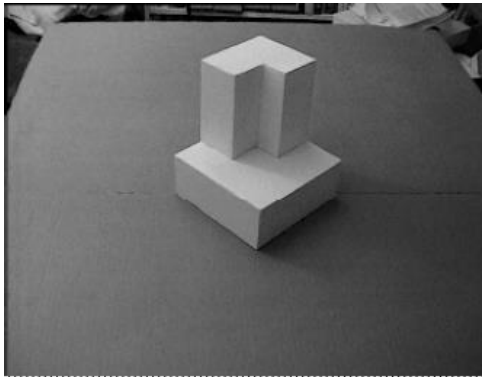
$$W(i,j) = \begin{bmatrix} 1 & 2 & 1 \\ 2 & \mathbf{3} & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- Weighting can be applied to non-rectangular filters
- Example: *cross-shaped* median filter may have weights

$$W^+(i,j) = \begin{bmatrix} 0 & 1 & 0 \\ 1 & \mathbf{1} & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

What is an Edge?

- Edge? sharp change in brightness (discontinuities)
- Where do edges occur?
 - Actual edges: Boundaries between objects
 - Sharp change in brightness can also occur within object
 - Reflectance changes
 - Change in surface orientation
 - Illumination changes. E.g. Cast shadow boundary

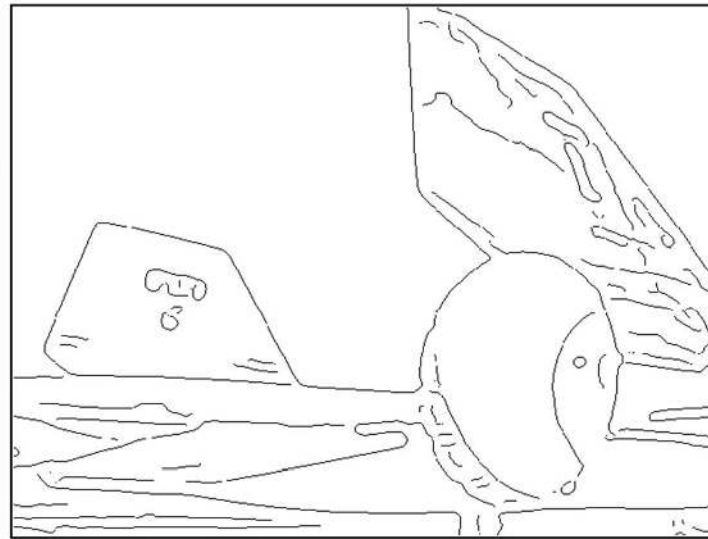


Edge Detection

- Image processing task that finds edges and contours in images
- Edges so important that human vision can reconstruct edge lines



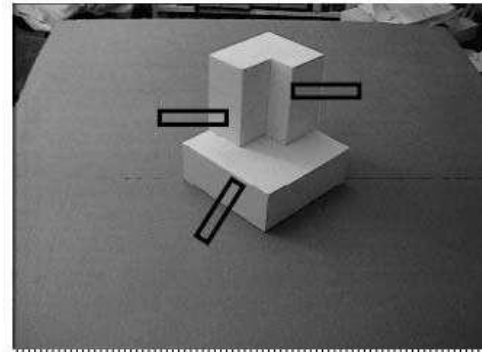
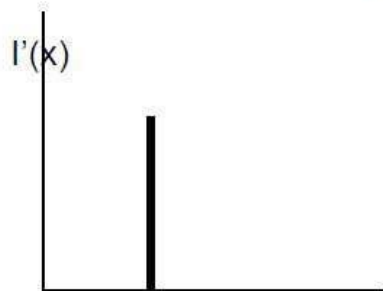
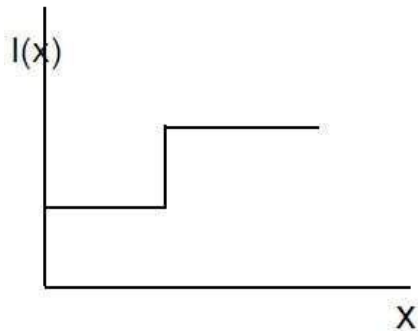
(a)



(b)

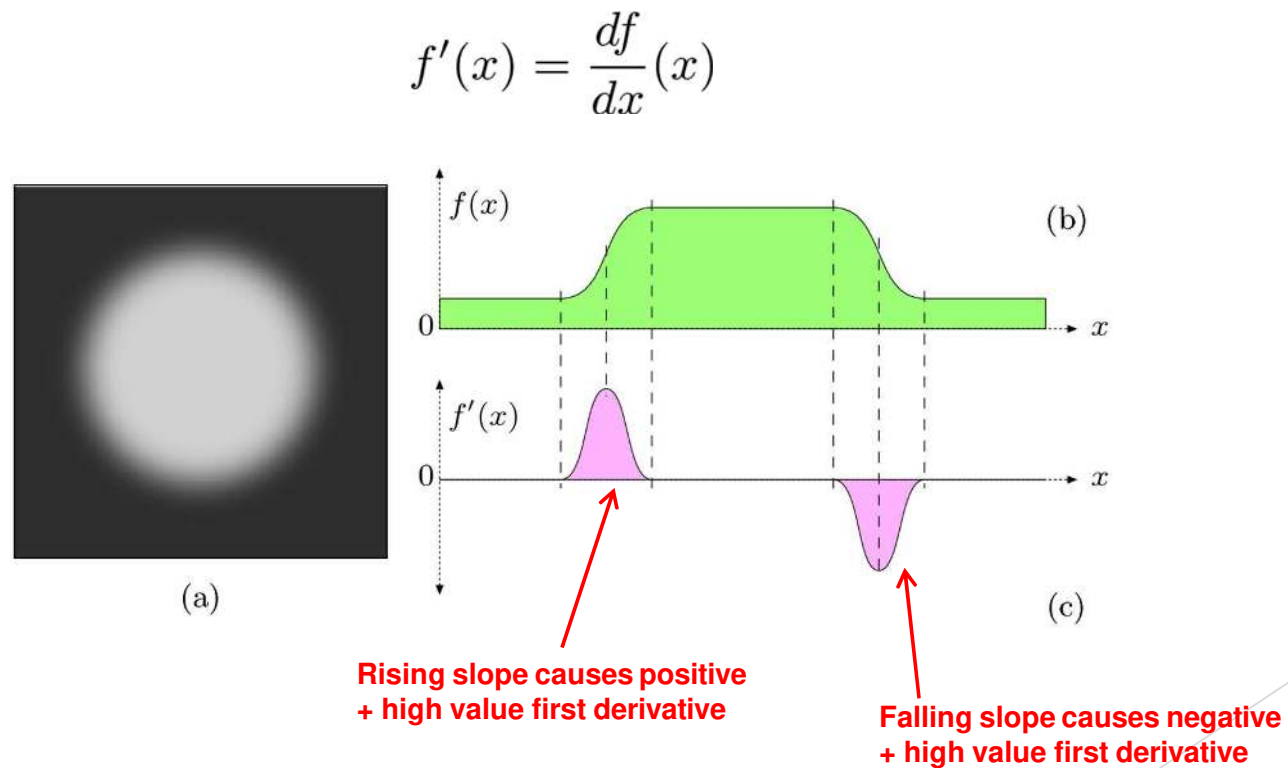
Characteristics of an Edge

- Edge: A sharp change in brightness
- Ideal edge is a step function in some direction



Characteristics of an Edge

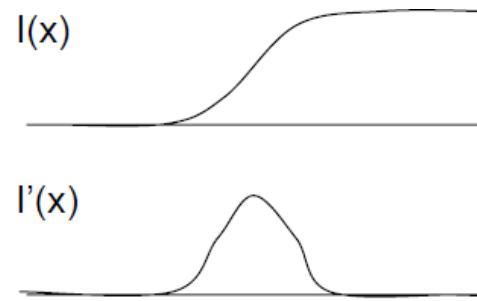
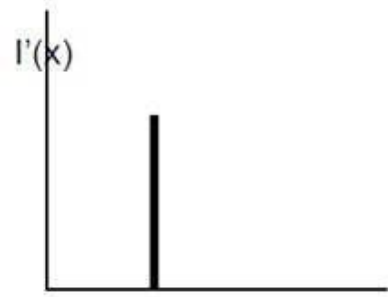
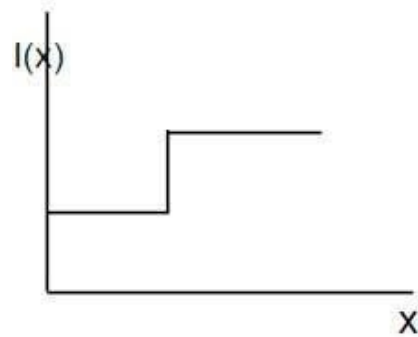
- Real (non-ideal) edge is a slightly blurred step function
- Edges can be characterized by high value first derivative



Characteristics of an Edge

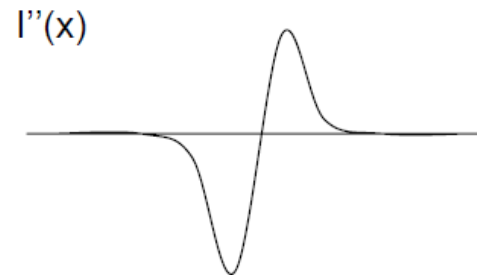
- Ideal edge is a step function in certain direction.
- First derivative of $I(x)$ has a peak at the edge
- Second derivative of $I(x)$ has a zero crossing at edge

Ideal edge



Real edge

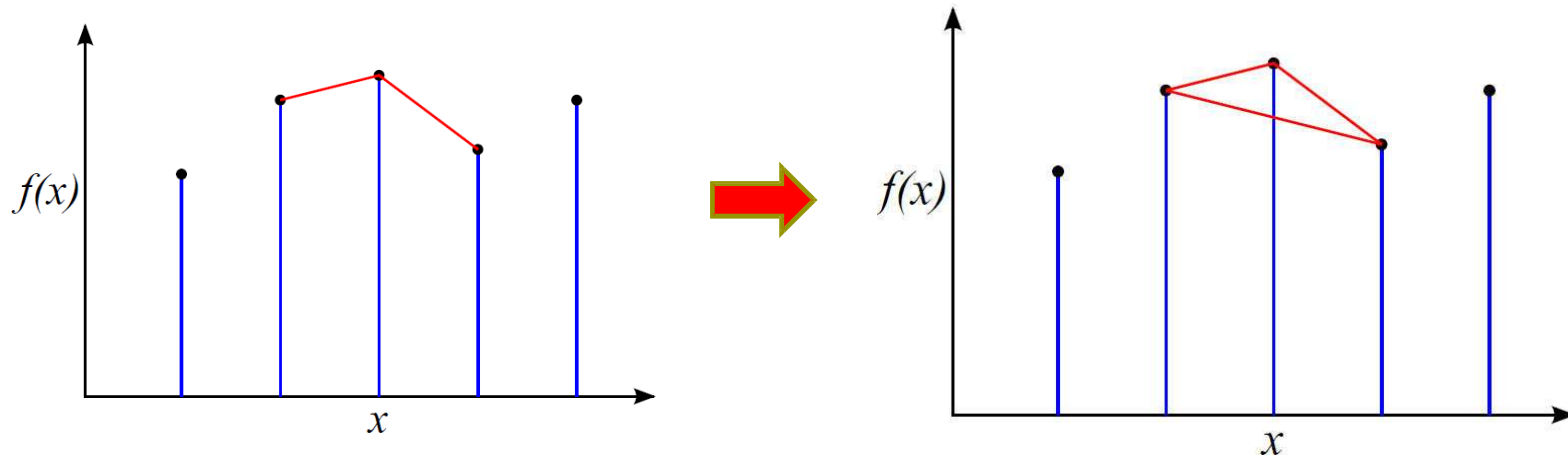
First derivative shows peak



Second derivative shows zero crossing

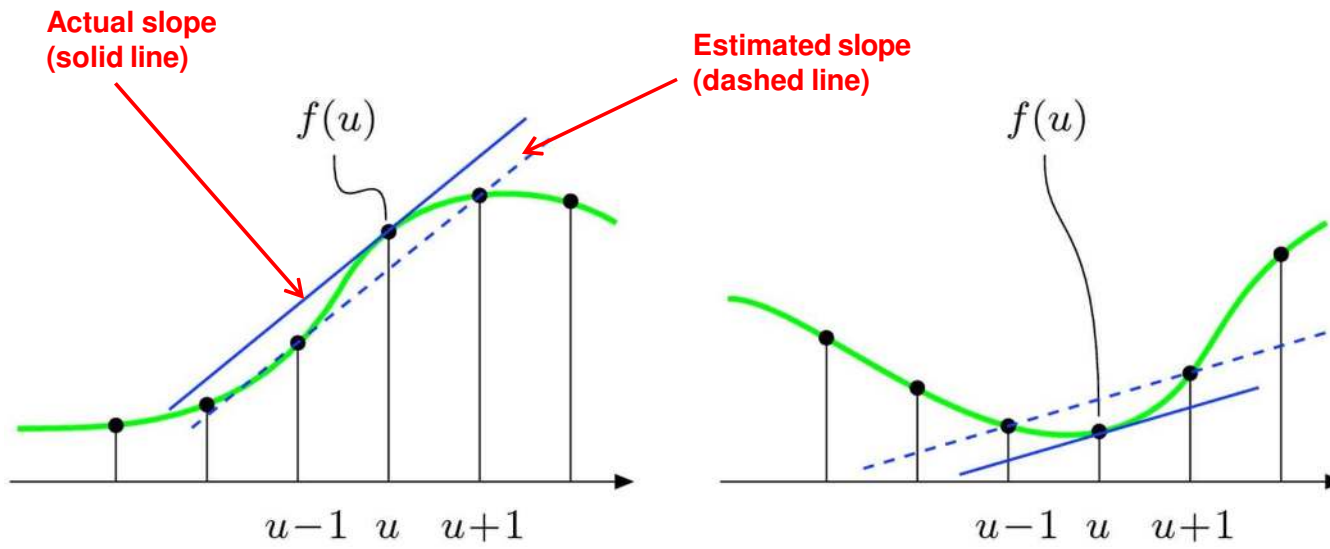
Slopes of Discrete Functions

- Left and right slope may not be same
- Solution? Take average of left and right slope



Computing Derivative of Discrete Function

$$\frac{df}{du}(u) \approx \frac{f(u+1) - f(u-1)}{2} = 0.5 \cdot (f(u+1) - f(u-1))$$



Finite Differences

- Forward difference (right slope)

$$\Delta_+ f(x) = f(x + 1) - f(x)$$

- Backward difference (left slope)

$$\Delta_- f(x) = f(x) - f(x - 1)$$

- Central Difference (average slope)

$$\Delta f(x) = \frac{1}{2} (f(x + 1) - f(x - 1))$$

Definition: Function Gradient



- Let $f(x,y)$ be a 2D function
- **Gradient:** Vector whose direction is in direction of maximum rate of change of f and whose magnitude is maximum rate of change of f
- Gradient is perpendicular to edge contour

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]^T$$

- magnitude = $\left[\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2 \right]^{1/2}$

- direction = $\tan^{-1} \left(\frac{\partial f / \partial y}{\partial f / \partial x} \right)$

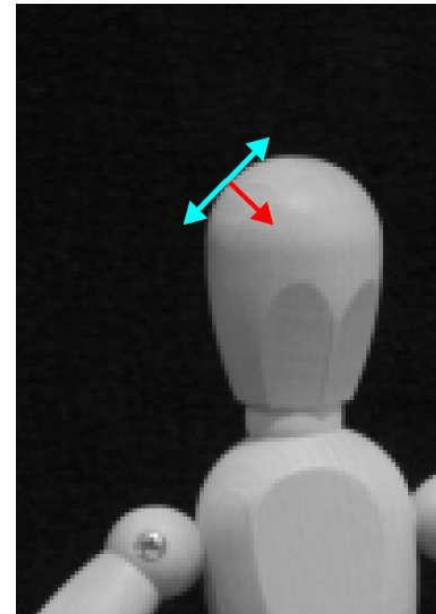


Image Gradient

- Image is 2D discrete function
- Image derivatives in horizontal and vertical directions

$$\frac{\partial I}{\partial u}(u, v) \quad \text{and} \quad \frac{\partial I}{\partial v}(u, v)$$

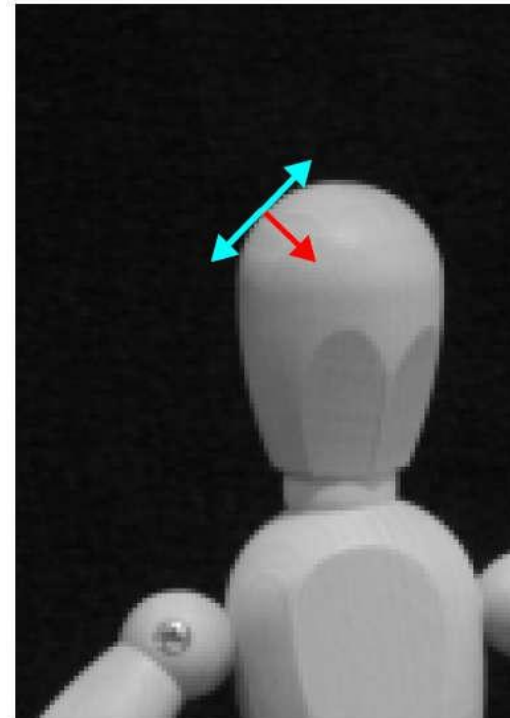
- Image gradient at location (u, v)

$$\nabla I(u, v) = \begin{bmatrix} \frac{\partial I}{\partial u}(u, v) \\ \frac{\partial I}{\partial v}(u, v) \end{bmatrix}$$

- Gradient magnitude

$$|\nabla I|(u, v) = \sqrt{\left(\frac{\partial I}{\partial u}(u, v)\right)^2 + \left(\frac{\partial I}{\partial v}(u, v)\right)^2}$$

- Magnitude is invariant under image rotation, used in edge detection



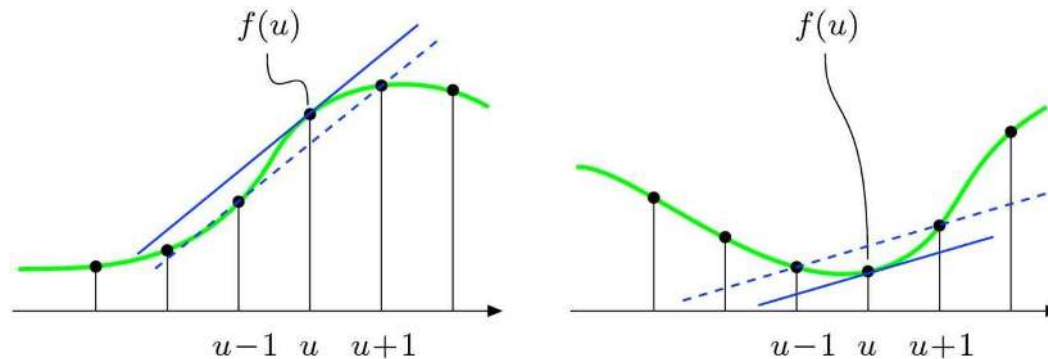
Derivative Filters

- Recall that we can compute derivative of discrete function as

$$\frac{df}{du}(u) \approx \frac{f(u+1) - f(u-1)}{2} = 0.5 \cdot (f(u+1) - f(u-1))$$

- Can we make linear filter that computes central differences

$$H_x^D = [-0.5 \quad 0 \quad 0.5] = 0.5 \cdot [-1 \quad 0 \quad 1]$$



Finite Differences as Convolutions

- Forward difference

$$\Delta_+ f(x) = f(x+1) - f(x)$$

- Take a convolution kernel $H = [0 \quad -1 \quad 1]$

$$\Delta_+ f = f * H$$

Finite Differences as Convolutions

- Central difference

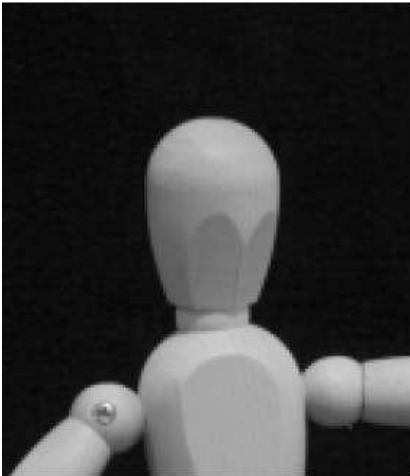
$$\Delta f(x) = \frac{1}{2} (f(x+1) - f(x-1))$$

- Convolution kernel is: $H = [-0.5 \quad 0 \quad 0.5]$

$$\Delta f(x) = f * H$$

- Notice: Derivative kernels sum to zero

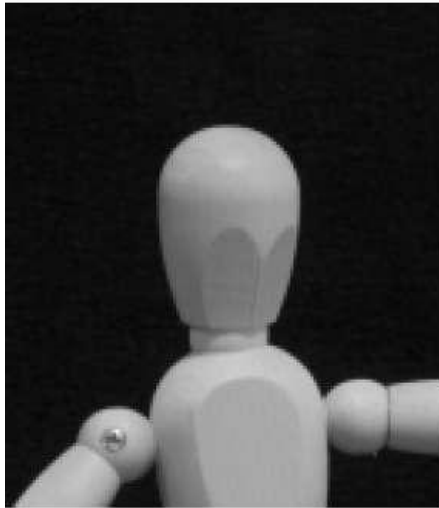
x-Derivative of Image using Central Difference



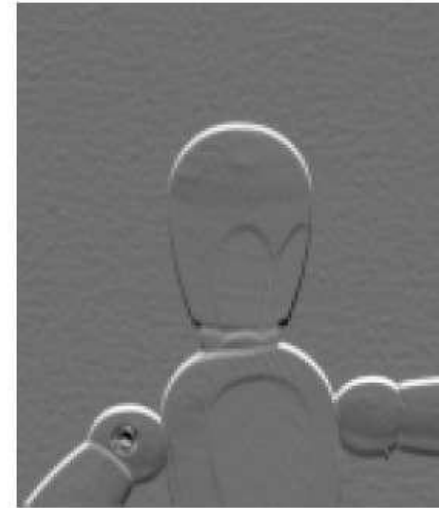
$$* \begin{bmatrix} -0.5 & 0 & 0.5 \end{bmatrix} =$$



y-Derivative of Image using Central Difference

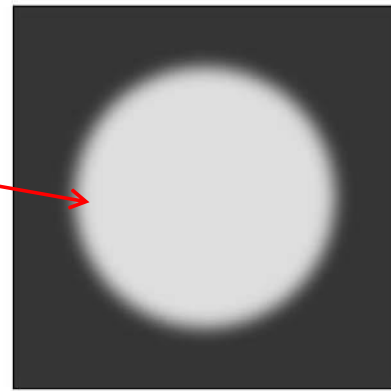


$$* \begin{bmatrix} -0.5 \\ 0 \\ 0.5 \end{bmatrix} =$$



Derivative Filters

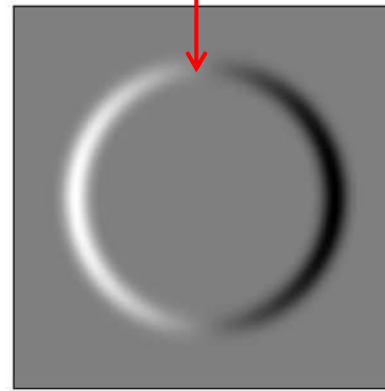
A synthetic image



(a)

Gradient slope in horizontal direction

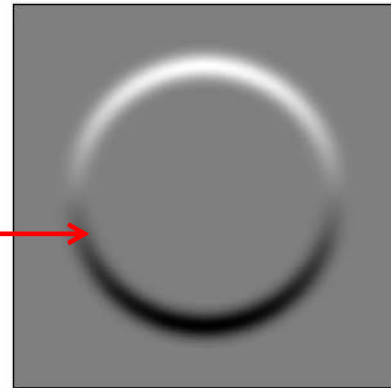
$$H_x^D = \begin{bmatrix} -0.5 & 0 & 0.5 \end{bmatrix} = 0.5 \cdot \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$$



(b)

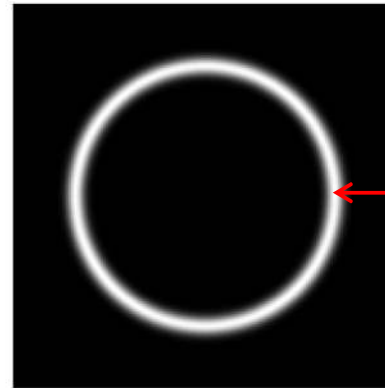
$$H_y^D = \begin{bmatrix} -0.5 \\ 0 \\ 0.5 \end{bmatrix} = 0.5 \cdot \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$

Gradient slope in vertical direction



(c)

Magnitude of gradient



(d)

Edge Operators

- Approximating local gradients in image is basis of many classical edge-detection operators
- Main differences?
 - Type of filter used to estimate gradient components
 - How gradient components are combined
- We are typically interested in
 - Local edge direction
 - Local edge magnitude

Partial Image Derivatives

- Partial derivatives of images replaced by finite differences

$$\Delta_x f = f(x, y) - f(x - 1, y) \quad \begin{bmatrix} -1 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

$$\Delta_y f = f(x, y) - f(x, y - 1)$$

- Alternatives are:

$$\Delta_{2x} f = f(x + 1, y) - f(x - 1, y) \quad \begin{bmatrix} -1 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$$

$$\Delta_{2y} f = f(x, y + 1) - f(x, y - 1)$$

- Robert's gradient

$$\Delta_+ f = f(x + 1, y + 1) - f(x, y) \quad \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

$$\Delta_- f = f(x, y + 1) - f(x + 1, y) \quad \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Prewitt

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Sobel

Using Averaging with Derivatives

- Finite difference operator is sensitive to noise
- Derivates more robust if derivative computations are averaged in a neighborhood
- Prewitt operator: derivative in x, then average in y

$$H_x^P = \frac{1}{3} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * [0.5 \quad 0 \quad -0.5] = \frac{1}{6} \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Average in y direction

Derivative in x direction

Note: Filter kernel is
flipped in convolution

- y-derivative kernel, H_y^P defined similarly

Sobel Operator

- Similar to Prewitt, but averaging kernel is higher in middle

$$H_x^S = \frac{1}{4} \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * [0.5 \quad 0 \quad -0.5] = \frac{1}{8} \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

$$H_y^S = \frac{1}{4} [1 \quad 2 \quad 1] * \begin{bmatrix} 0.5 \\ 0 \\ -0.5 \end{bmatrix} = \frac{1}{8} \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Average in x direction

Derivative in y direction

Note: Filter kernel is flipped in convolution

Prewitt and Sobel Edge Operators

- Prewitt Operator

$$H_x^P = \begin{bmatrix} -1 & 0 & 1 \\ -1 & \mathbf{0} & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad H_y^P = \begin{bmatrix} -1 & -1 & -1 \\ 0 & \mathbf{0} & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Written in separable form $\rightarrow H_x^P = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} * \begin{bmatrix} -1 & \mathbf{0} & 1 \end{bmatrix}$ and $H_y^P = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} * \begin{bmatrix} -1 \\ \mathbf{0} \\ 1 \end{bmatrix}$

- Sobel Operator

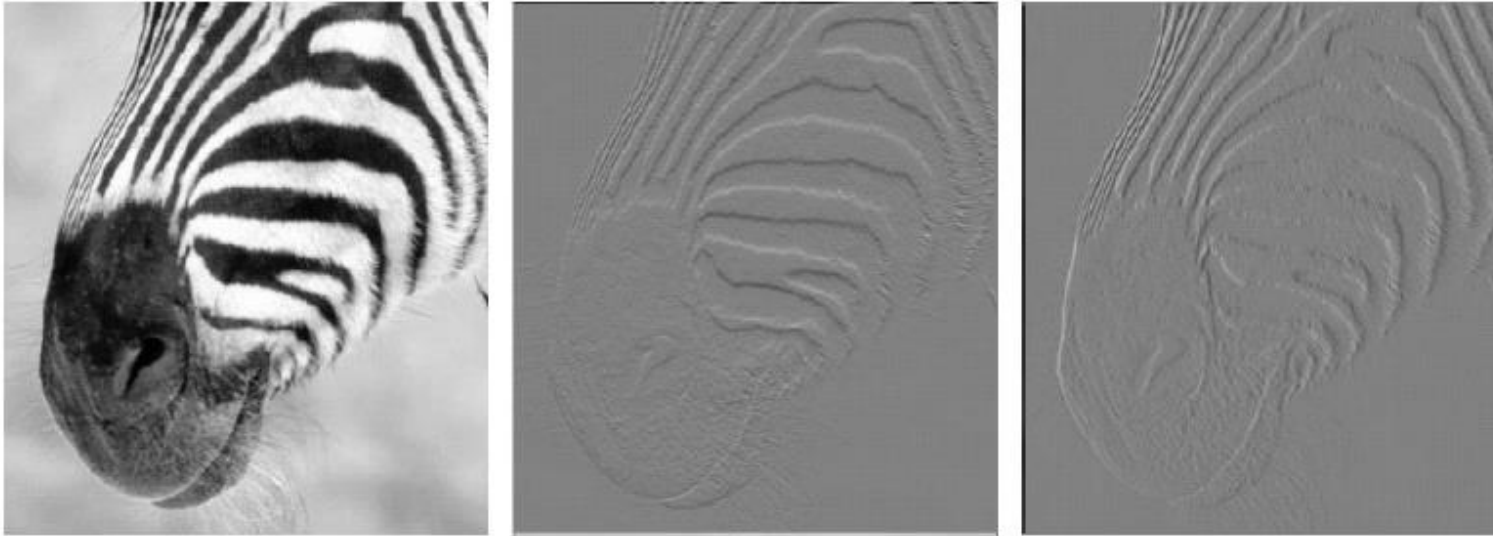
$$H_x^S = \begin{bmatrix} -1 & 0 & 1 \\ -2 & \mathbf{0} & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad H_y^S = \begin{bmatrix} -1 & -2 & -1 \\ 0 & \mathbf{0} & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Improved Sobel Filter

- Original Sobel filter relatively inaccurate
- Improved versions proposed by Jahne

$$H_x^{S'} = \frac{1}{32} \begin{bmatrix} -3 & 0 & 3 \\ -10 & \mathbf{0} & 10 \\ -3 & 0 & 3 \end{bmatrix} \quad \text{and} \quad H_y^{S'} = \frac{1}{32} \begin{bmatrix} -3 & -10 & -3 \\ 0 & \mathbf{0} & 0 \\ 3 & 10 & 3 \end{bmatrix}$$

Prewitt and Sobel Edge Operators



Scaling Edge Components

- Estimates of local gradient components obtained from filter results by appropriate scaling

Scaling factor for Prewitt operator $\longrightarrow \nabla I(u, v) \approx \frac{1}{6} \cdot \begin{bmatrix} (I * H_x^P)(u, v) \\ (I * H_y^P)(u, v) \end{bmatrix}$

Scaling factor for Sobel operator $\longrightarrow \nabla I(u, v) \approx \frac{1}{8} \cdot \begin{bmatrix} (I * H_x^S)(u, v) \\ (I * H_y^S)(u, v) \end{bmatrix}$

Gradient-Based Edge Detection

- Compute image derivatives by convolution

$$D_x(u, v) = H_x * I \quad \text{and} \quad D_y(u, v) = H_y * I$$

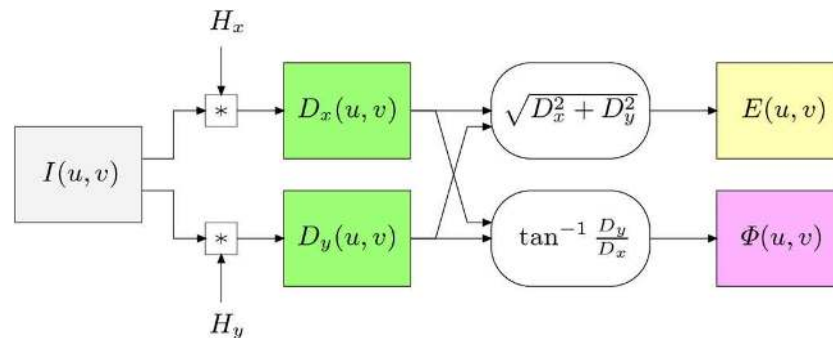
Scaled Filter results

- Compute edge gradient magnitude

$$E(u, v) = \sqrt{(D_x(u, v))^2 + (D_y(u, v))^2}$$

- Compute edge gradient direction

$$\Phi(u, v) = \tan^{-1} \left(\frac{D_y(u, v)}{D_x(u, v)} \right) = \text{ArcTan}(D_x(u, v), D_y(u, v))$$



Typical process of
Gradient based
edge detection

Gradient-Based Edge Detection

- After computing gradient magnitude and orientation then what?
- Mark points where gradient magnitude is large wrt neighbors



References

- Wilhelm Burger and Mark J. Burge, Digital Image Processing, Springer, 2008
- University of Utah, CS 4640: Image Processing Basics, Spring 2012
- Rutgers University, CS 334, Introduction to Imaging and Multimedia, Fall 2012